KU LEUVEN

Lecture 2: Supervised learning and kNN

Machine learning and inductive inference

Sieben Bocklandt

Slides based on slidedecks from Jesse Davis' lectures in Principles of Machine Learning

Goals of this lecture

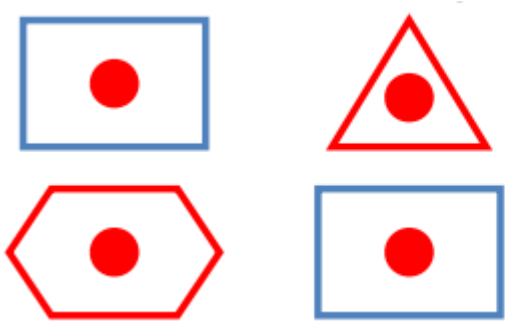
- Understand the basic goal of inductive supervised learning
- Understand the use of feature construction
- Know and understand what a hypothesis is
- Understand the components of a learning algorithm

- Understand why kNN is an important algorithm
- Know how kNN, distance and similarity measures and decision surfaces work
- Know how kNN forms a decision surface
- Understand pitfalls with kNN and how to address them

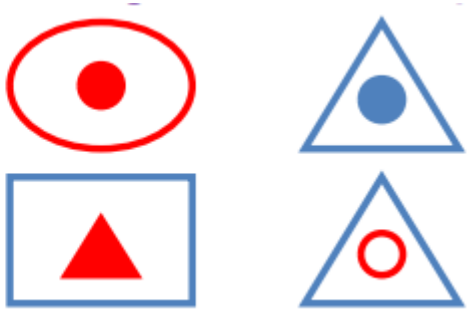
Supervised learning

Example problem

Positive examples

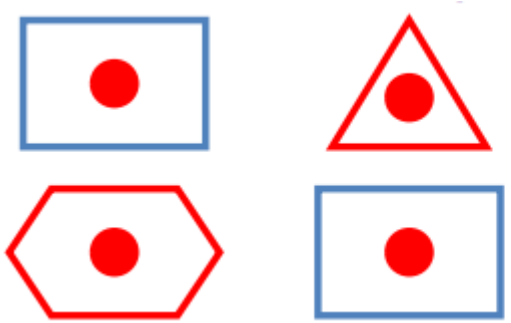


Negative examples

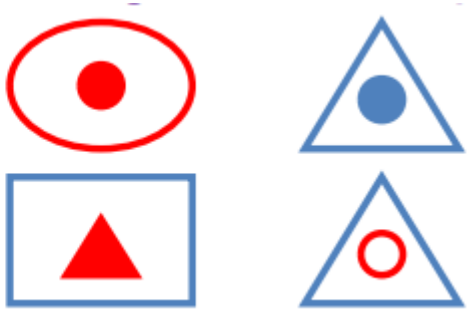


Example problem

Positive examples



Negative examples

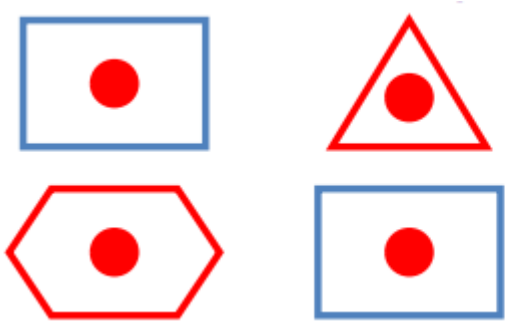


How do we classify following example?

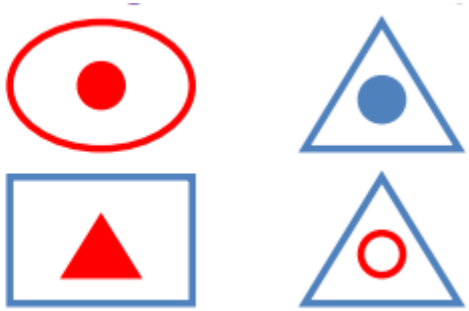


Example problem

Positive examples



Negative examples



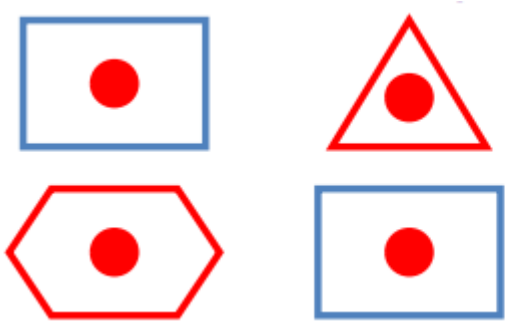
How do we classify following example?



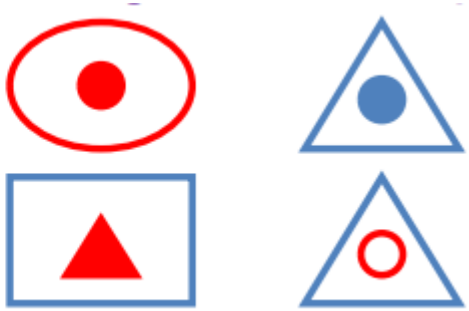
+ Solid red circle in a polygon

Example problem

Positive examples



Negative examples



How do we classify following example?

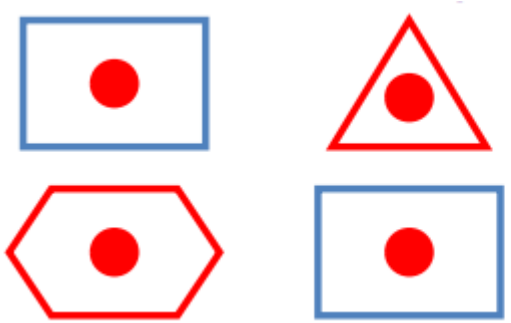


+ Solid red circle in a polygon

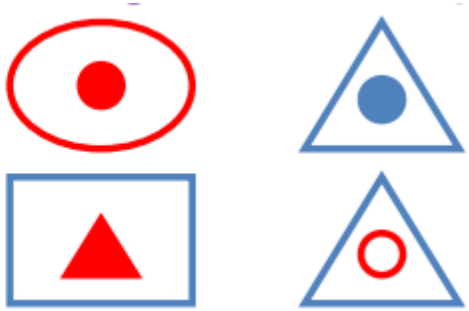
- Solid red circle in a regular polygon

Example problem

Positive examples



Negative examples



How do we classify following example?



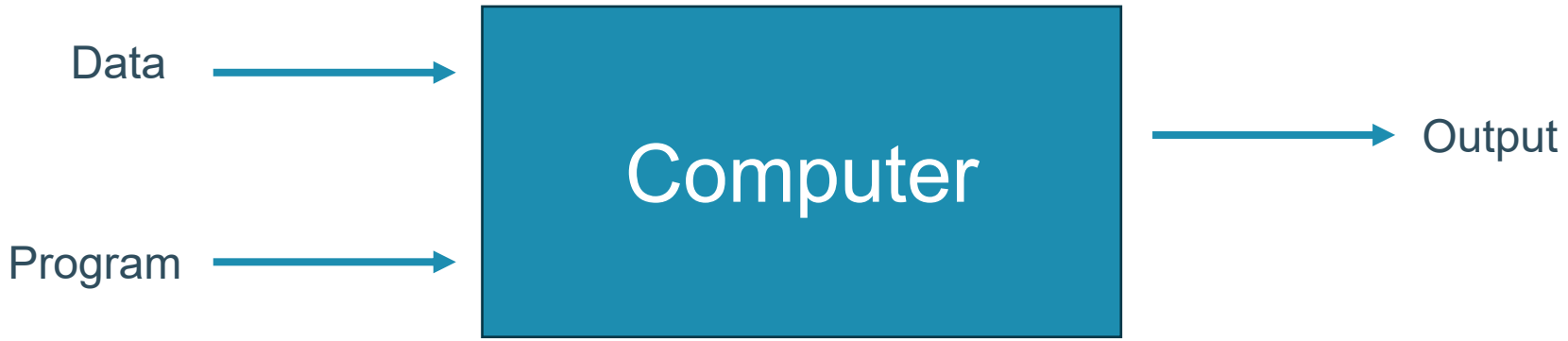
+ Solid red circle in a polygon

- Solid red circle in a regular polygon

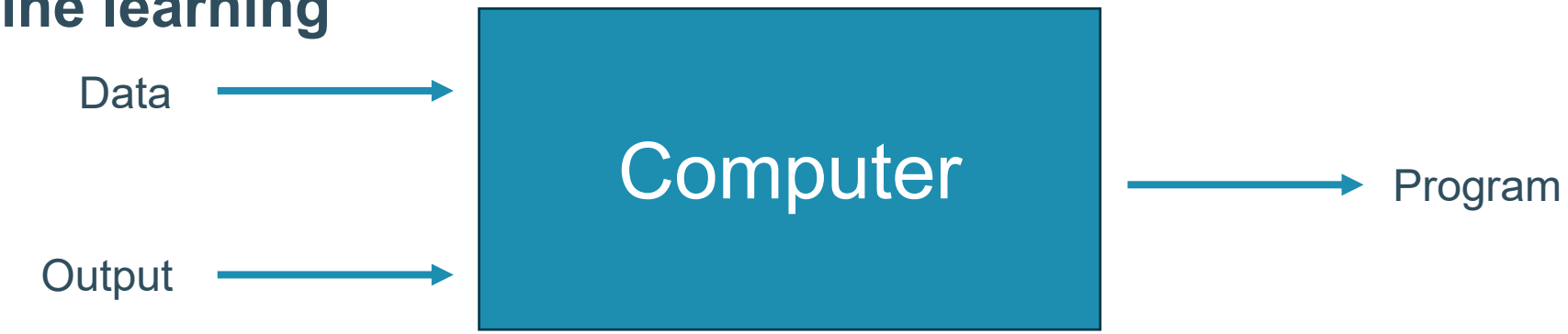
…

Programming vs Supervised learning

Traditional programming

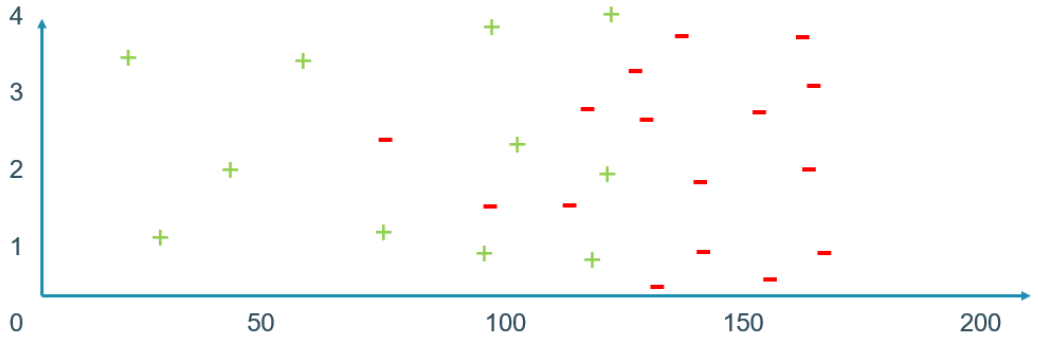


Machine learning



Supervised learning

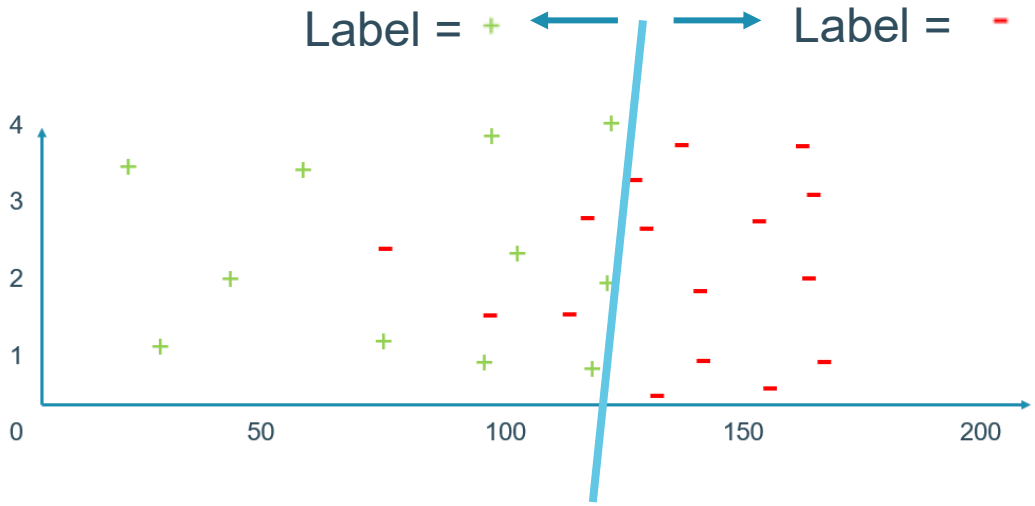
- **Given** a set of data consisting of < *x* , *y* > with
  - *x* a set of features
  - *y* a label associated with *x*



Supervised learning

- **Given** a set of data consisting of < *x* , *y* > with
  - *x* a set of features
  - *y* a label associated with *x*

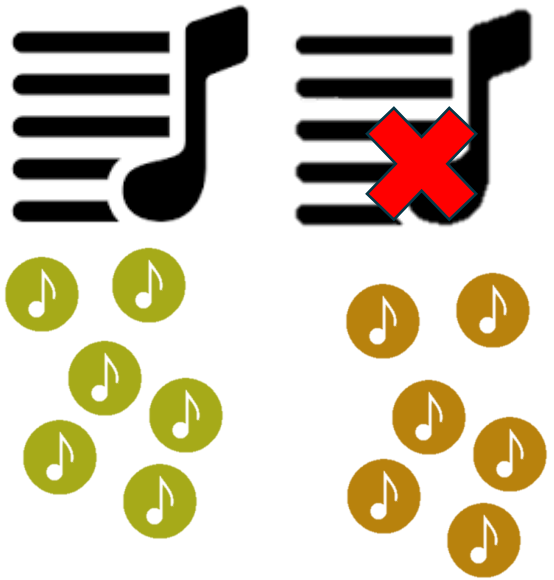
- **Learn** a model / program / hypothesis that
  - for a new set of features *x'*
  - says how to map *x'* to an output *y'*



How to do supervised learning

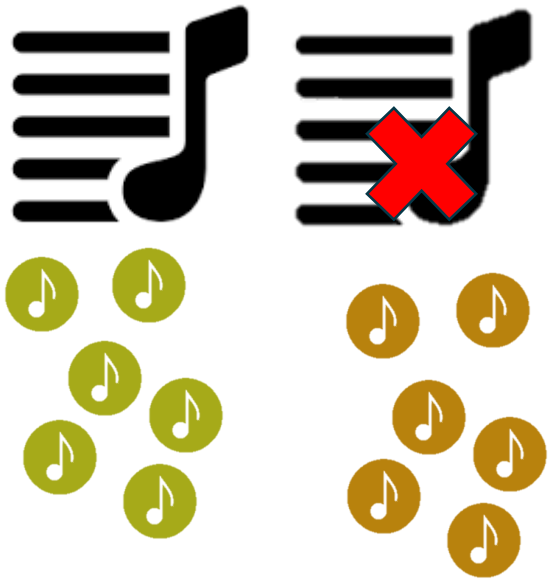
Running example

- Music playlist consisting of songs used as background music



Running example

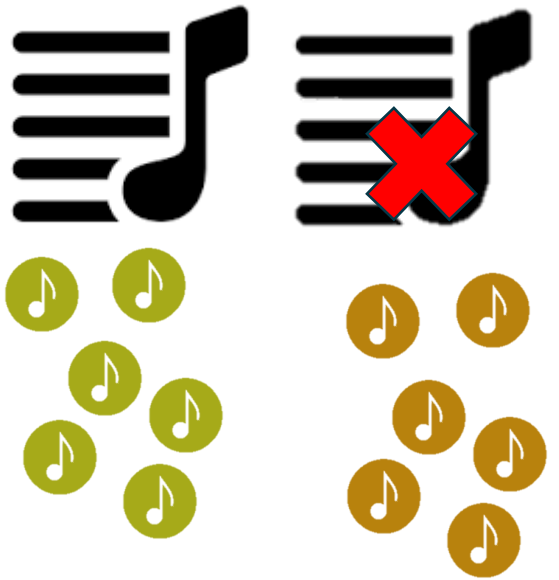
- Music playlist consisting of songs used as background music
- We want to automatically add new songs





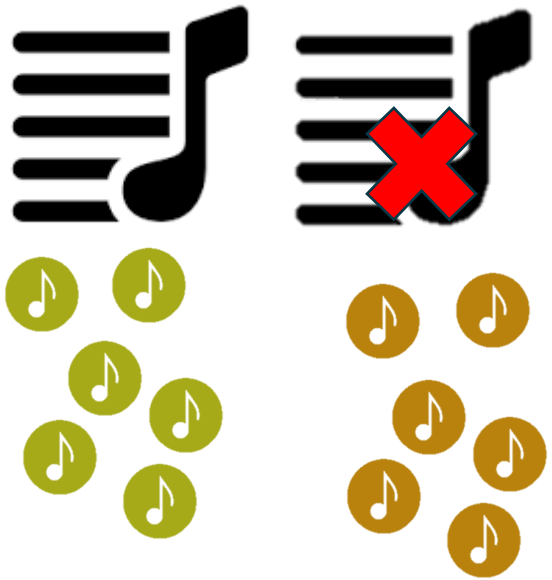
Running example

- Music playlist consisting of songs used as background music
- We want to automatically add new songs
- What do we need to do to to solve this problem?





Running example



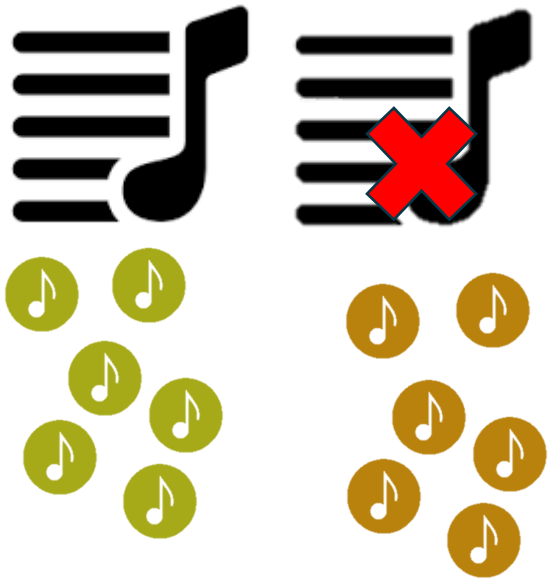
- Music playlist consisting of songs used as background music
- We want to automatically add new songs
- What do we need to do to to solve this problem?

1. Data collection
2. Data preprocessing
3. Feature construction
4. Model building
5. Model evaluation



Running example

- Music playlist consisting of songs used as background music
- We want to automatically add new songs
- What do we need to do to to solve this problem?



1. Data collection
2. Data preprocessing
3. Feature construction
4. Model building
5. Model evaluation

Focus in practice

Focus of this class / academic research



Data collection

- What to collect? (Images, Sensor readings, Documents, …) → Structured?
- Where to collect? (Websites, IoT sensors, Databases, …) → Primary data / third party?
- How to collect? (Scraping, API's, Queries, Document processing, …) → Allowed?
- How much to collect? (Memory, #Examples, Regulations, …) → Needed vs practical?
- Where to store the collected data? (Data warehouse, Data lake, Data mart) → Speed?

Data Preprocessing

- Evaluating, filtering, manipulating and encoding data to make sure the ML model can work with it
- Examples:
  - Missing data (Remove example, add mean value, …)
  - Scaling/normalizing (Multiple variables can have different scales)
  - Outliers (Remove examples, alter variable value, …)
  - Encode data (non-numerical → numerical)
  - Divide into train-validation-test (Train the model, tune the model, evaluate the model)

Feature construction

- Which features can describe a song?

Feature construction

- Which features can describe a song?
  - Genre
  - Duration
  - Tempo (BPM)
  - Key
  - Loudness (dB)
  - Acousticness
  - Energy
  - Instrumental
  - …

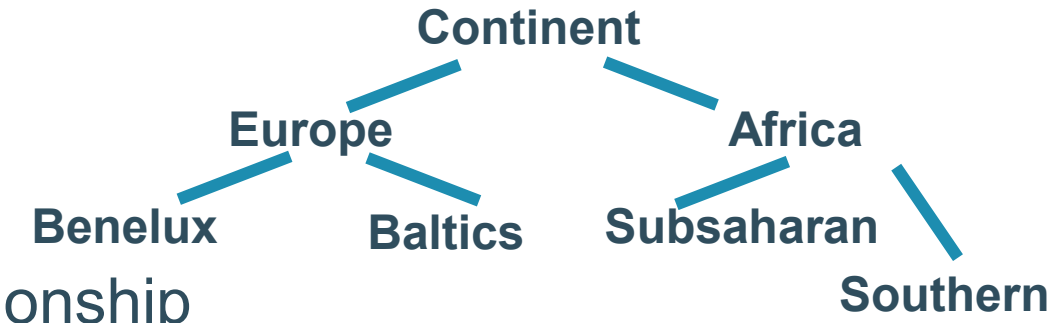
Defining features

- Features describe the problem
- Called many different names
  - Features (Machine learning)
  - Attributes (Machine learning / databases)
  - Covariates (Statistical learning)
  - Variables (Probabilistic learning)
  - KPIs (Business / applications)

- Defining features is an art and skill (no ground truth)

Types of Features

- Boolean (1/0 ; True/False)
  - E.g. Instrumental={True, False}
- Nominal: No relationship between values
  - E.g. Genre = {Pop, Rock, Blues, Jazz, Punk}
- Linear/Ordered: Feature values are ordered
  - E.g. Energy = {Low,Medium,High}
- Numerical
  - E.g. BPM= 1..250
- Hierarchical: Partial ordering with an IS-A relationship



Fixed length feature vector

Features: properties that describe the problem

- Given a new data point *d* (song, image, document, …) 

Fixed length feature vector

Features: properties that describe the problem

- Given a new data point *d* (song, image, document, …) 
- Define *m* features

Title	Genre	BPM	Key	…	Instrumental	

Fixed length feature vector

Features: properties that describe the problem

- Given a new data point *d* (song, image, document, …) 
- Define *m* features
- Each feature has v_i possible values (domain based on type)

Title	Genre	BPM	Key	…	Instrumental	Belongs to playlist
String	Nominal	Numerical	Nominal		Boolean	Boolean

Fixed length feature vector

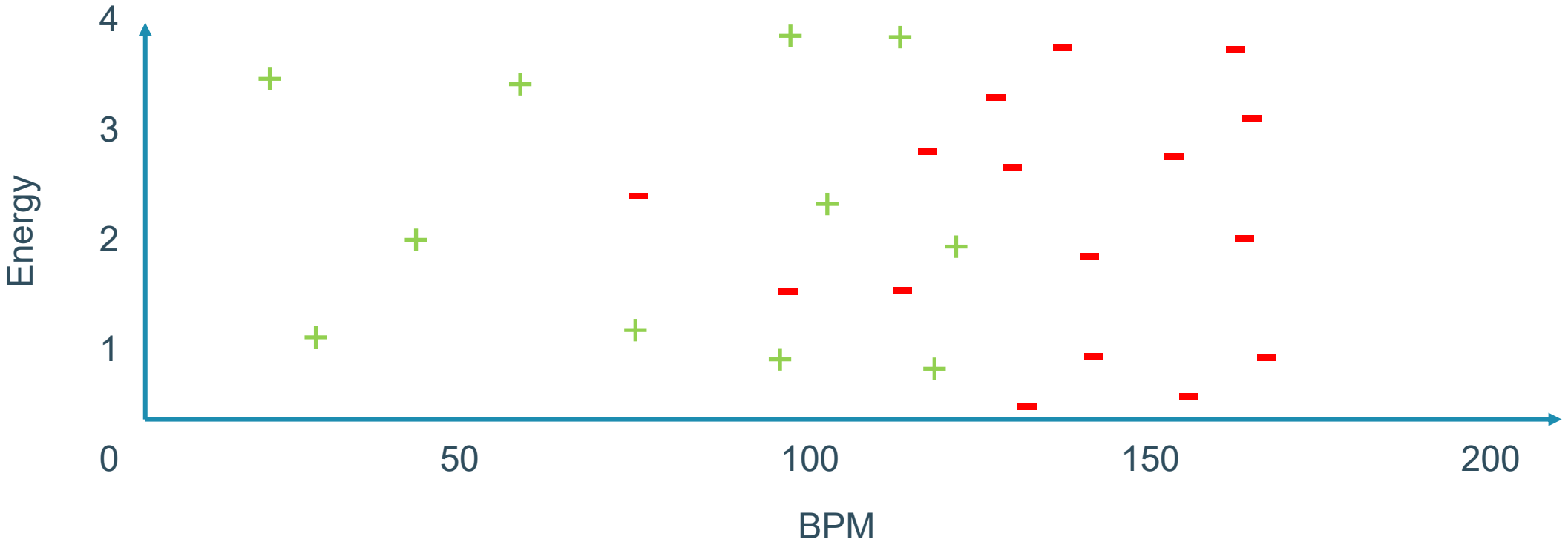
Features: properties that describe the problem

- Given a new data point *d* (song, image, document, …) 
- Define *m* features
- Each feature has v_i possible values (domain based on type)
- Each example is represented by a vector of *m* feature values and (possibly) a class label

Title	Genre	BPM	Key	…	Instrumental	Belongs to playlist
String	Nominal	Numerical	Nominal		Boolean	Boolean
Mamma mia	Pop	137	D major	…	No	No

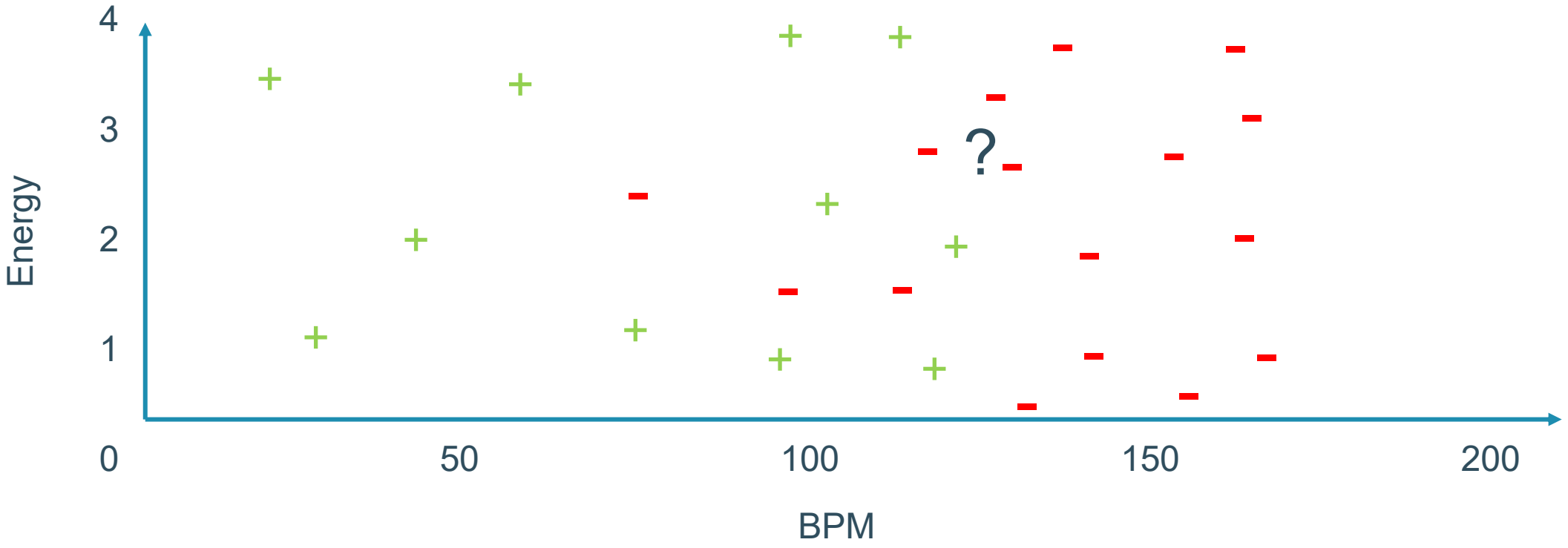
Feature Space Visually

- Each axis is a feature with its own domain. This example is simplified (only 2 features)



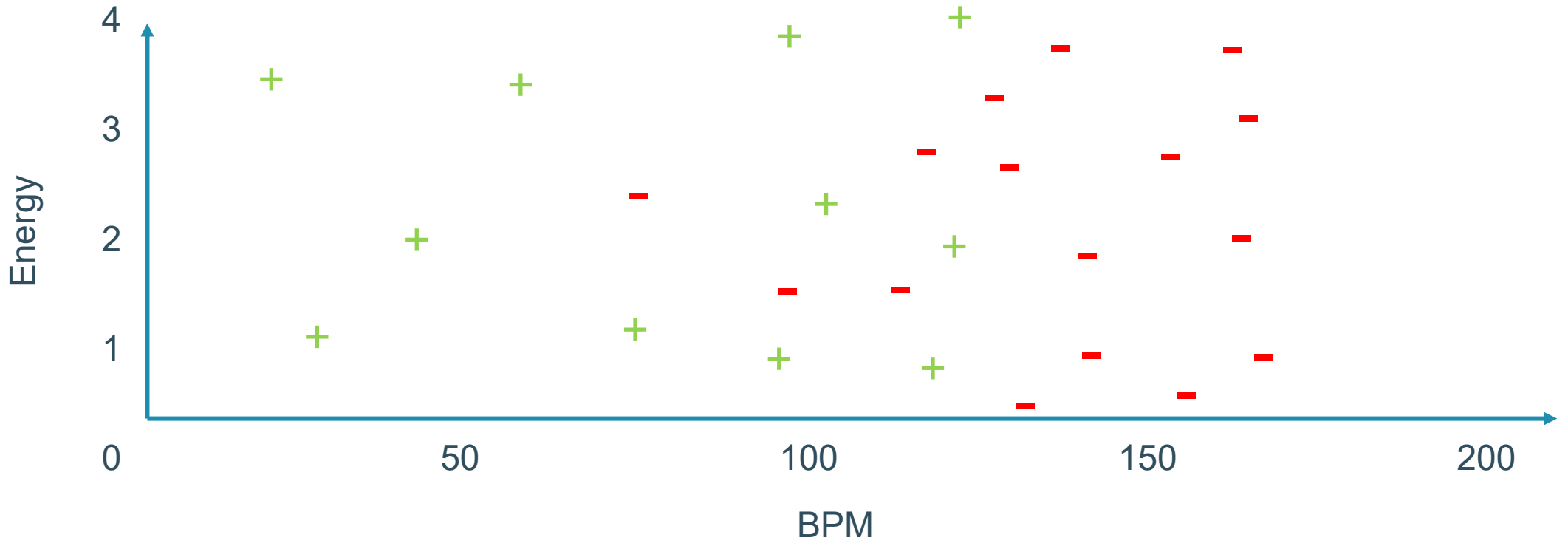
Feature Space Visually

- Each axis is a feature with its own domain. This example is simplified (only 2 features)
- A new song (BPM=125, Energy=2.7) belongs in the feature space

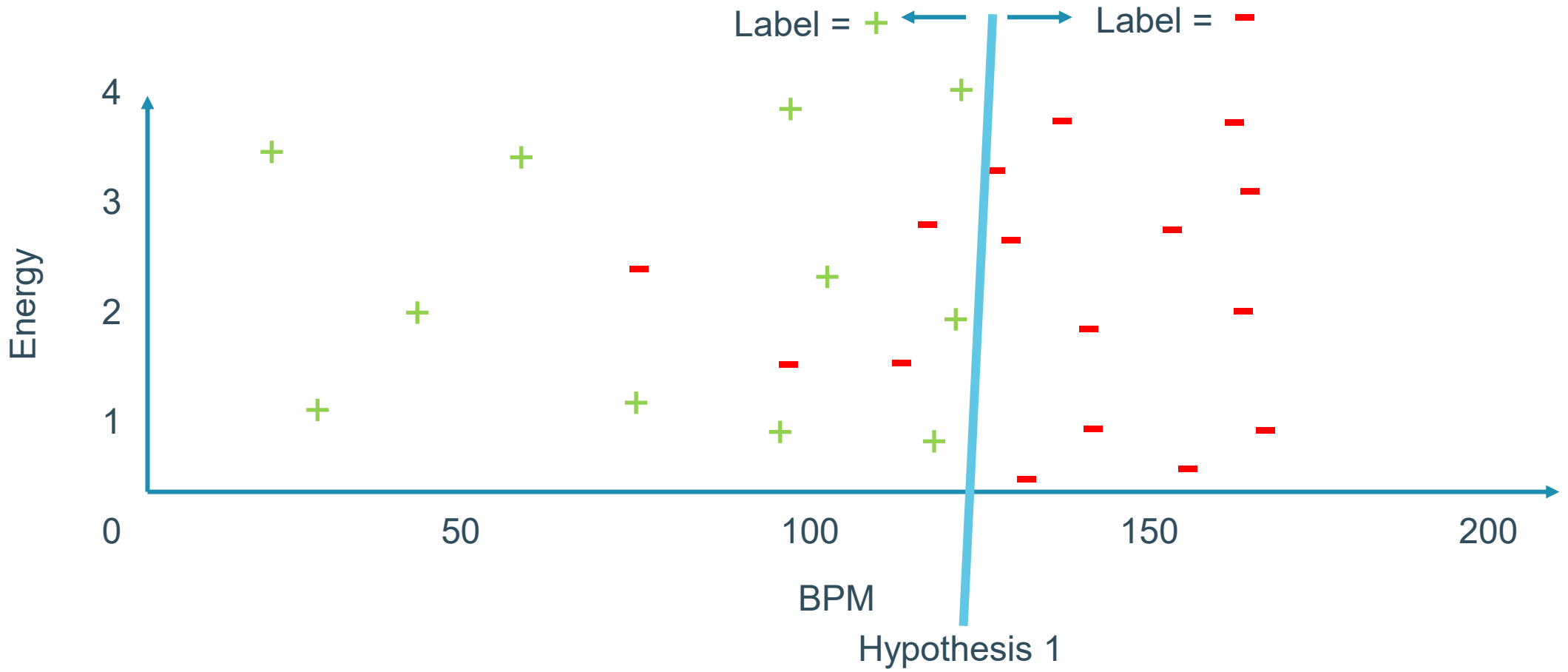


Representing models

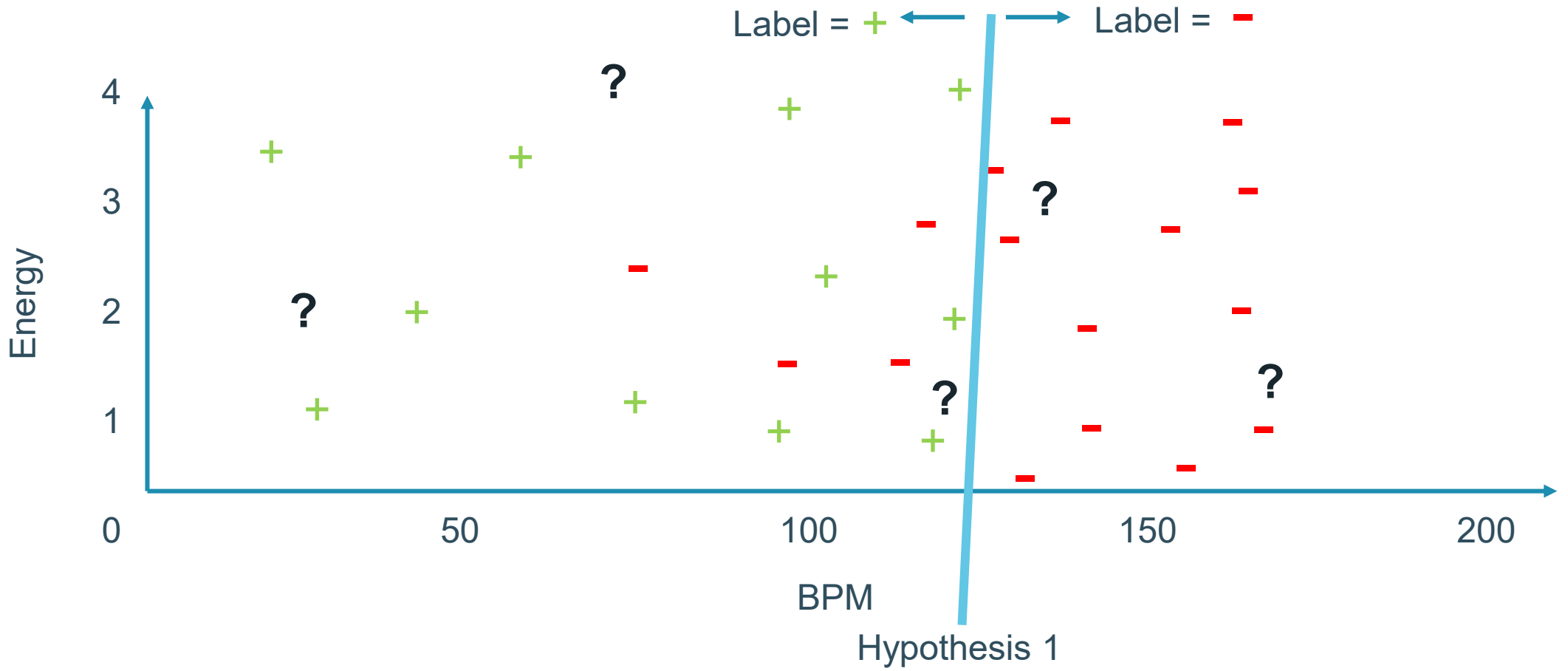
Hypothesis space



Hypothesis space

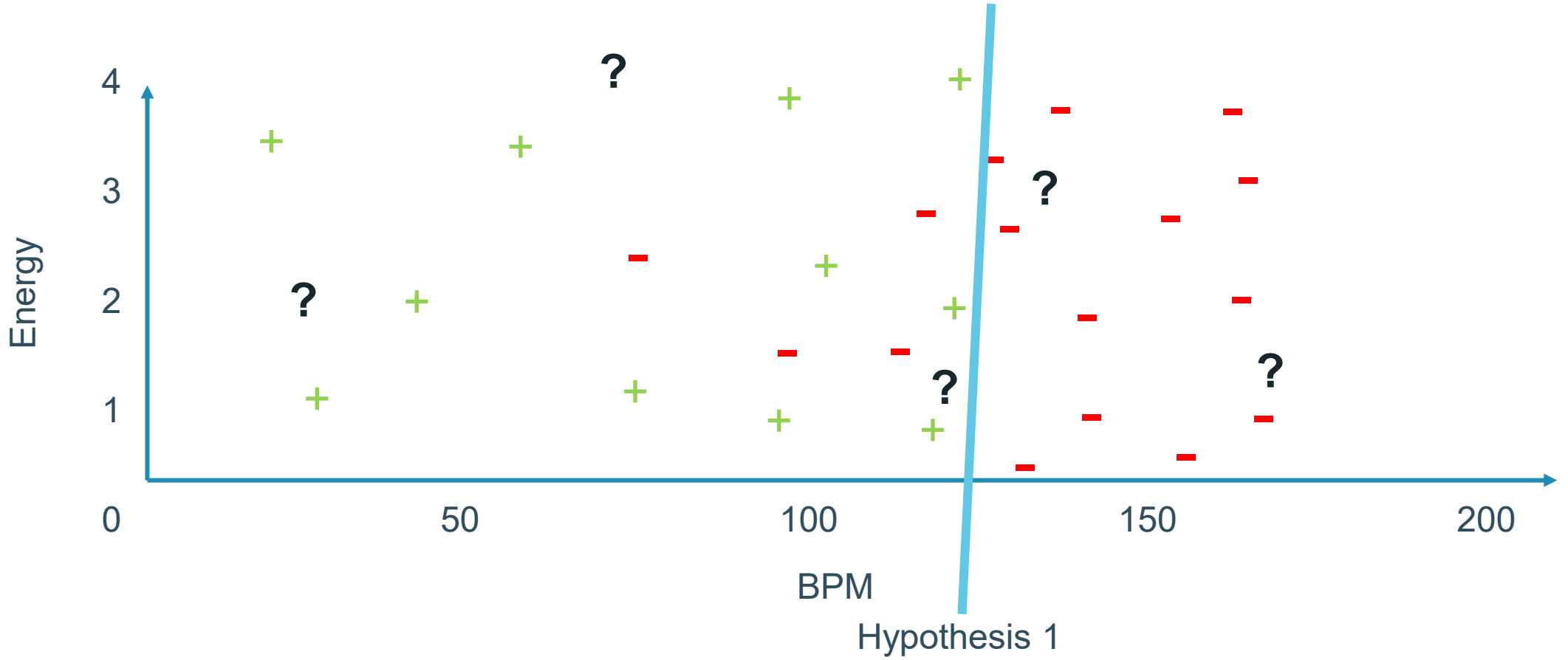


Hypothesis space



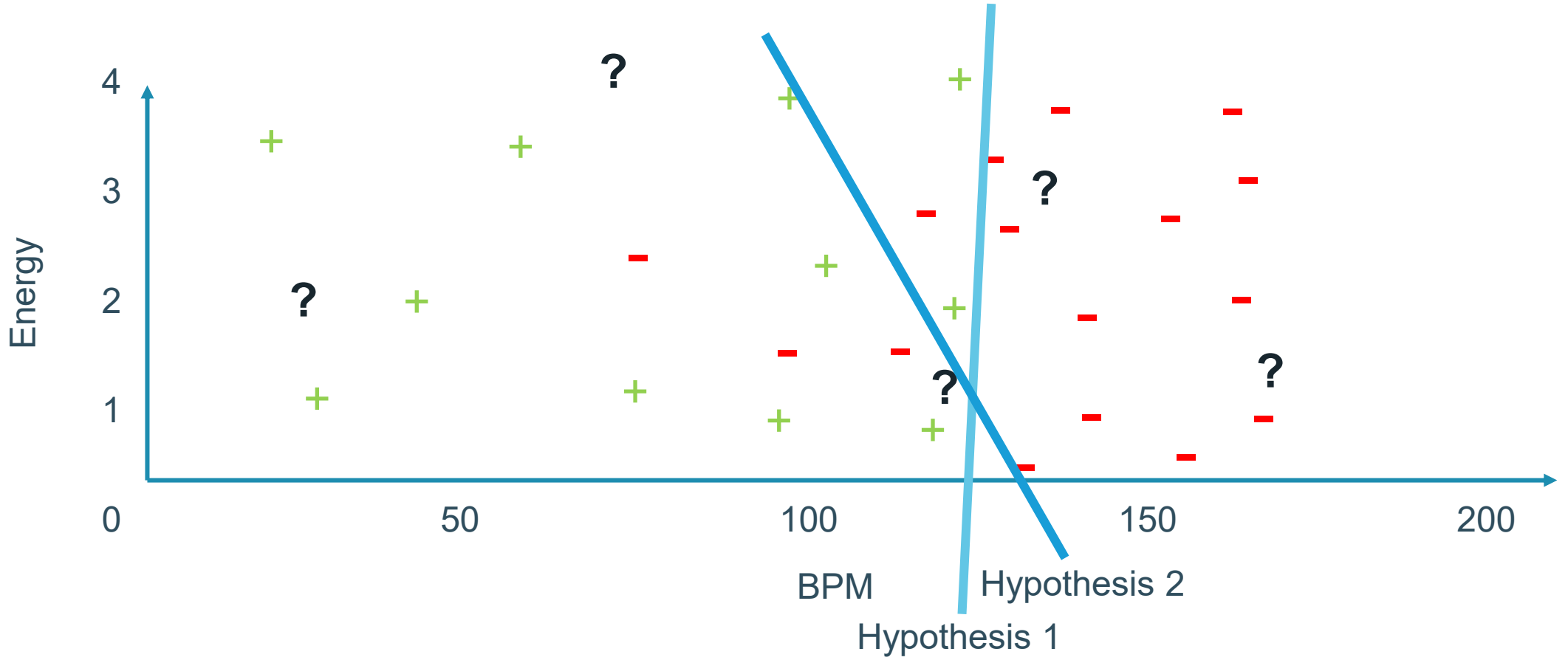
Hypothesis space

Set of legal hypotheses: in this example only straight lines



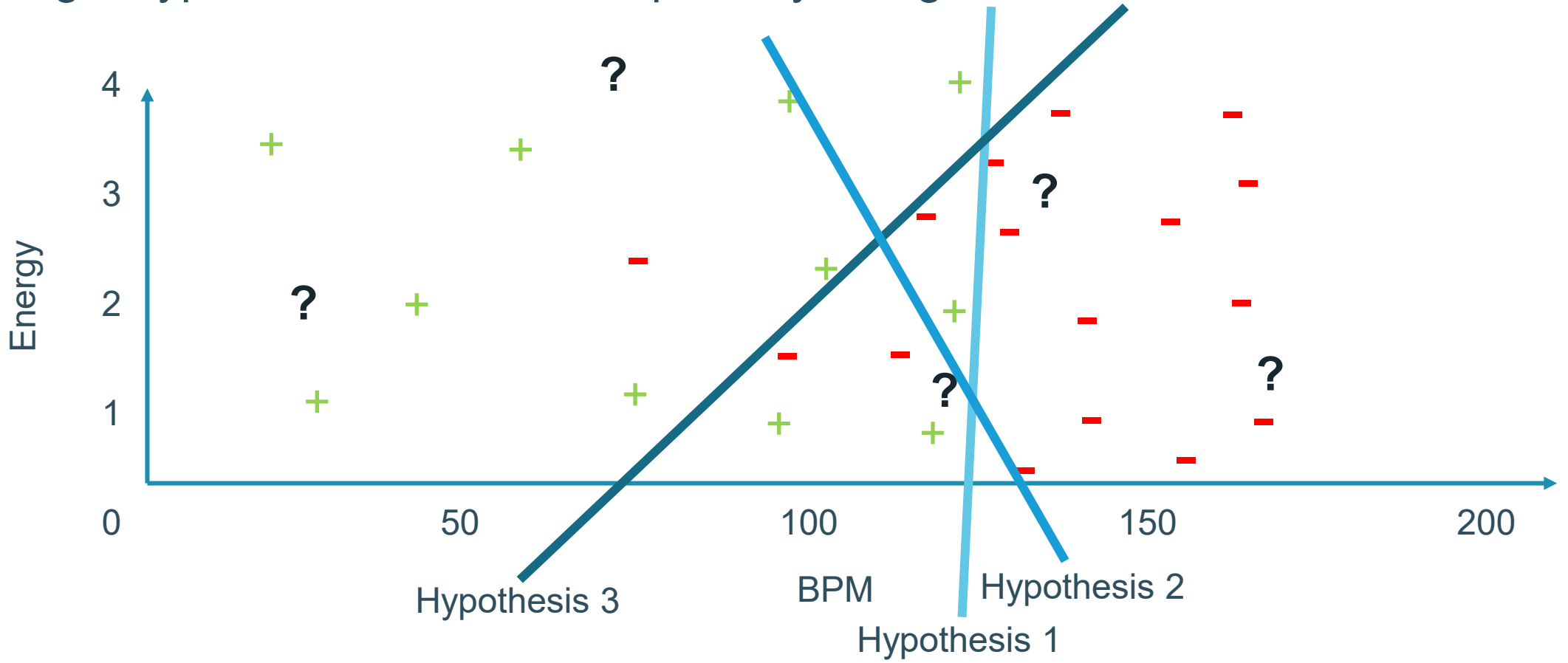
Hypothesis space

Set of legal hypotheses: in this example only straight lines



Hypothesis space

Set of legal hypotheses: in this example only straight lines

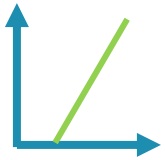


Fixed size hypothesis classes

- Assume a fixed form for the relationship between the features and the target prior to seeing data

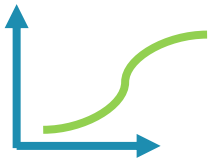
Fixed size hypothesis classes

- Assume a fixed form for the relationship between the features and the target prior to seeing data
- Canonical examples include
  - Linear regression:



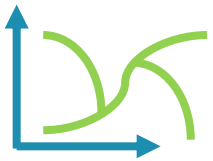
$$F(x) = w_1X_1 + w_2X_2 + w_3X_3 + ... + w_dX_d$$

  - Logistic regression:



$$P(Y = 1 \mid x, w) = \frac{\exp(w_0 + \sum_d w_j \times x_d)}{1 + \exp(w_0 + \sum_d w_j \times x_d}$$

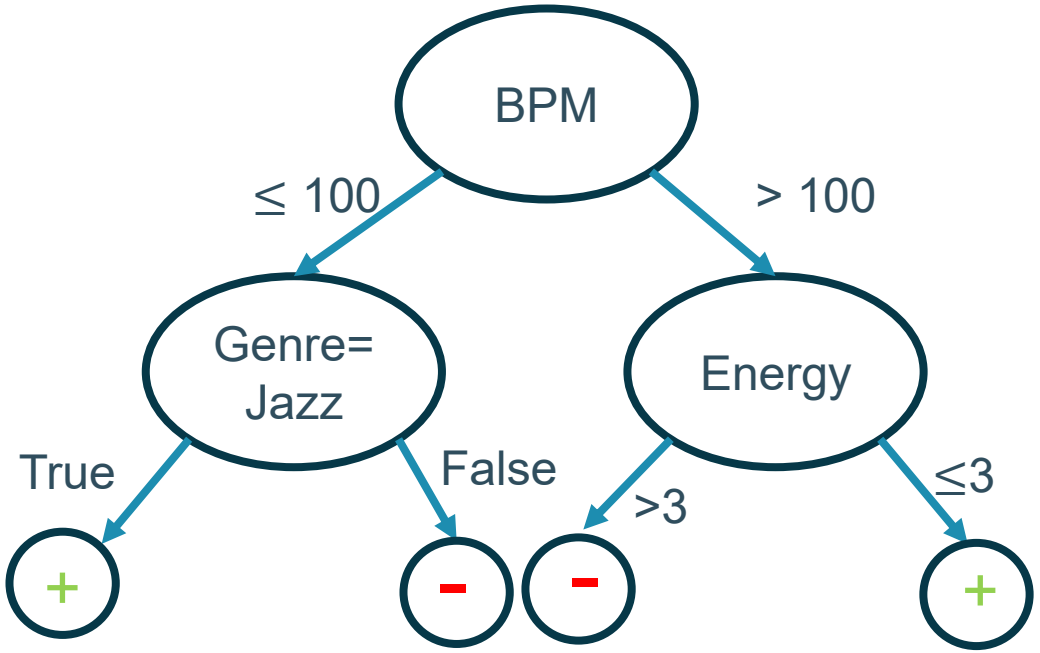
  - Naïve Bayes:



$$\text{argmax}_{c \in Y} P(Y = c) \prod_d P(X_d = x_d | Y = c)$$

Variable size hypothesis classes

- Structure and form of the model changes during learning, often becoming more complex when you have more data
- Canonical examples include
  - Rule sets
  - Decision trees



Learning a model

Overall learning objective for binary classification

- **Given**
  - Set of positive examples
  - Set of negative examples

  ⊢ Drawn independently at random from some unknown probability distribution

Overall learning objective for binary classification

- **Given**
  - Set of positive examples
  - Set of negative examples

  ⊢ Drawn independently at random from some unknown probability distribution

- **Learn:** A model that applies to
  - All/most of the positive examples
  - None/few of the negative examples
  - **Correctly labels most future examples**

Note: easy to extend this definition to general classification (#classes > 2)

Fundamental challenge of learning

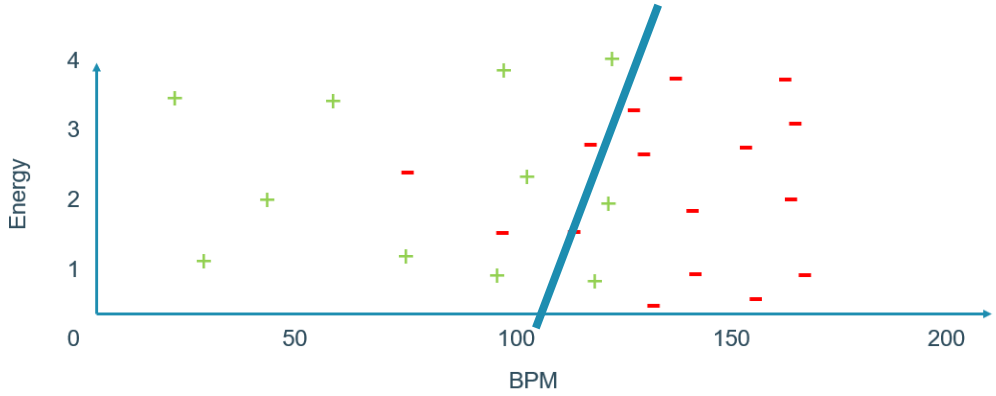
- Suppose I have a problem with 4 binary features and 2 classes
  - Possible examples = 2 x 2 x 2 x 2 = 16
  - Possible ways to label= 2 x 2 x … x 2 = 2^{16} = 65536

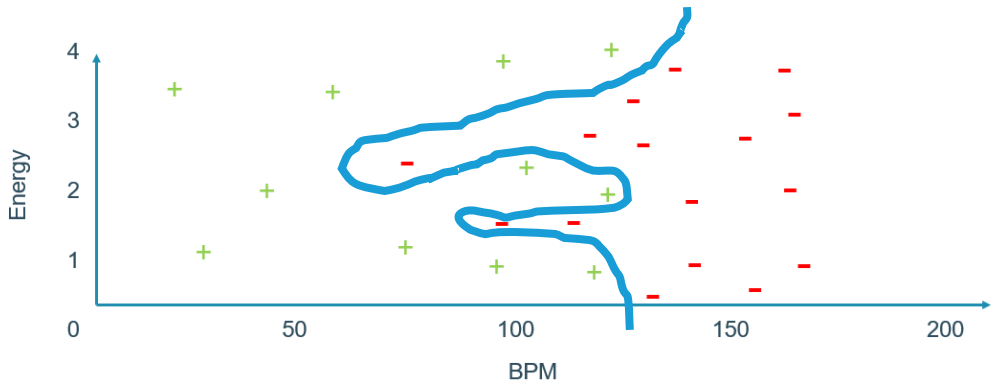
- Observing part of the data does not rule out much
  - E.g. We observe 7 labeled examples, this leaves 9 unobserved examples
    → 2^9 possible labelings still left

- How can we decide which model is best?

Solution: Inductive Bias

- We need to make assumptions: experience alone doesn't allow us to make conclusions about unseen data instances
- Two types of bias
  - **Restriction:** Limit the hypothesis space (Rules, straight lines, parabolas, …)
  - **Preference:** Impose an ordering on the hypothesis space (e.g. Simpler models)





Goals of learning

- At high-level, algorithms learn one of these:
  - Function that predicts class or real value (e.g. decision tree, linear regression)
  - Conditional probability distribution P(Y|X) (e.g. logistic regression, neural network)
  - Joint probability distribution P(X,Y) (e.g. Naïve Bayes)

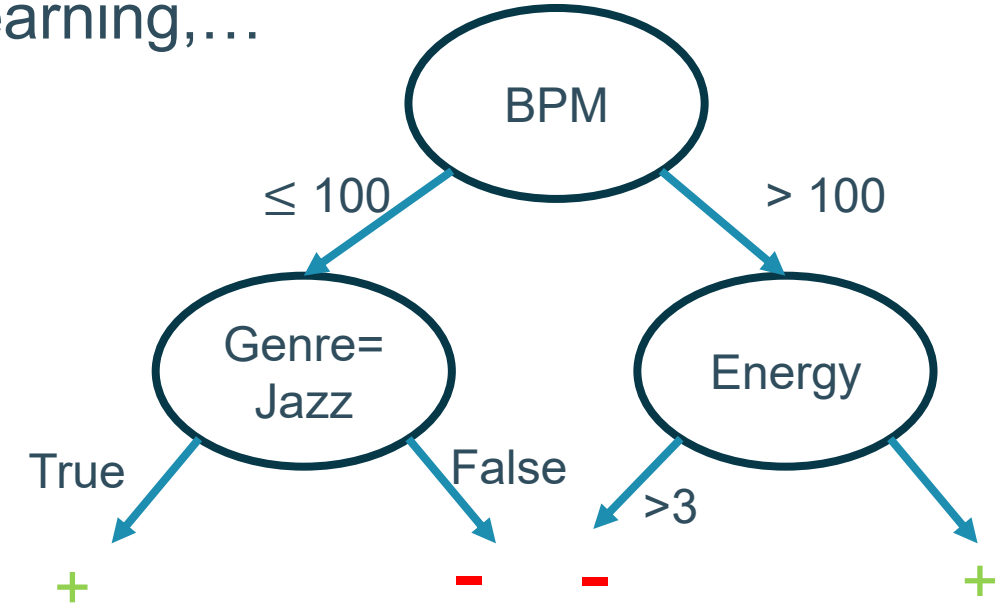
Goals of learning

- At high-level, algorithms learn one of these:
  - Function that predicts class or real value (e.g. decision tree, linear regression)
  - Conditional probability distribution P(Y|X) (e.g. logistic regression, neural network)
  - Joint probability distribution P(X,Y) (e.g. Naïve Bayes)

- Learning a model involves a 2-step approach
  - Searching through the hypothesis space
  - Evaluating candidate hypotheses (and stop the search when good enough)

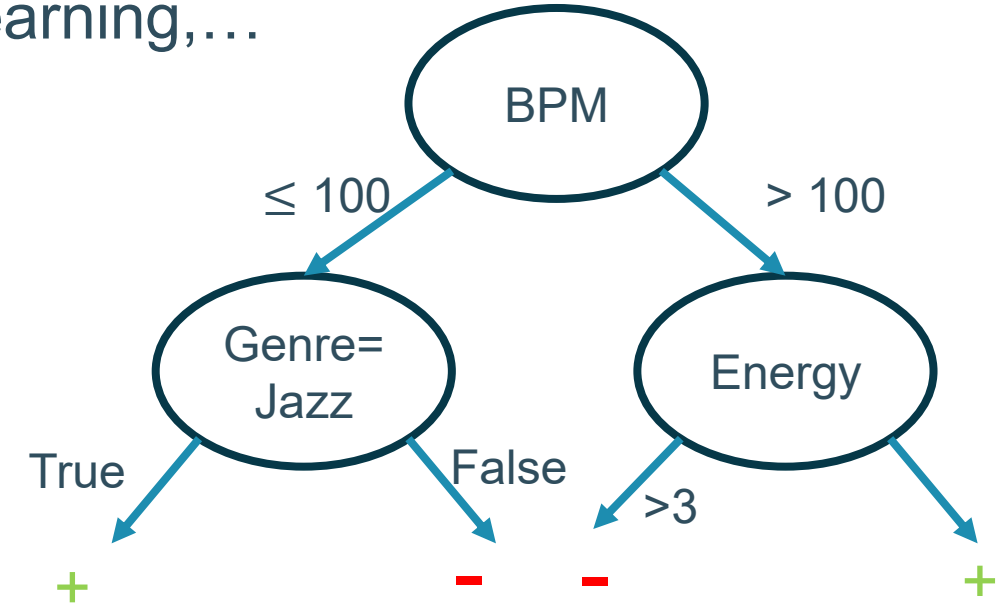
Searching through the hypothesis space

- **Discrete:**
  - What model should I build?
  - Combinatorial search (e.g. best first, depth first, beam-search, …)
  - Hyperparameter choices, architecture, …
  - E.g. Decision tree, feature selection, rule learning,…



Searching through the hypothesis space

- **Discrete:**
  - What model should I build?
  - Combinatorial search (e.g. best first, depth first, beam-search, …)
  - Hyperparameter choices, architecture, …
  - E.g. Decision tree, feature selection, rule learning,…
- **Continuous:**
  - What numbers should this model contain?
  - Optimization (e.g. gradient descent)
  - Weight tuning, parameter tuning, …
  - E.g. Neural networks



Evaluating candidate models

- Typically we have two high-level components:
  - How well does the model fit the data?
  - How “complex” is the model?

Evaluating candidate models

- Typically we have two high-level components:
  - How well does the model fit the data?
  - How “complex” is the model?

- Tradeoff fit (good) for complexity (bad)
  - In an ideal case: perfect fit without complexity
  - E.g. rule learning: #positives covered - #negatives covered - #literals
    - $x_1 \vee x_2 \vee x_3 \Longrightarrow Y$ #P = 20 #N = 10
    - $(x_1 \vee x_3) \wedge (x_2 \vee x_4) \wedge (x_1 \vee x_2) \Longrightarrow Y$ #P = 21 #N = 9

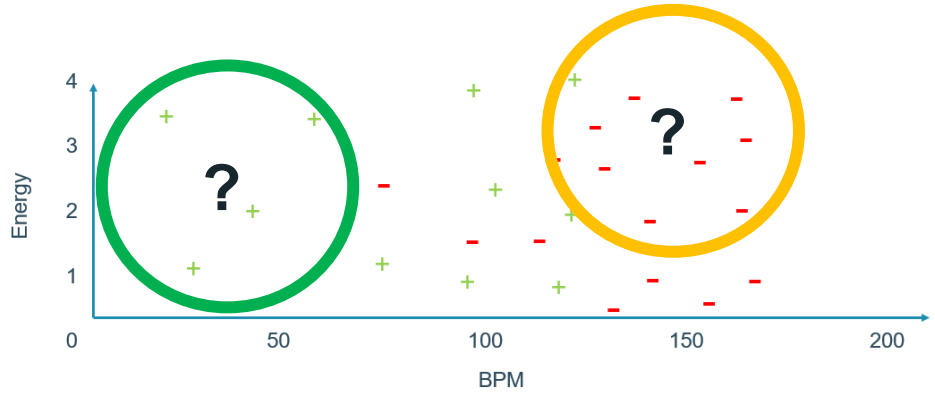
Evaluation of a model often framed in terms of loss

- A loss function quantifies the impact or consequence of an error (incorrect prediction)
  - E.g. 0/1-loss is 0 if correct, 1 if incorrect.
    The average 0/1-loss over a dataset is the classification error rate.
- Goal is to find
  - a hypothesis *h in hypothesis space* $\mathcal{H}$
  - that minimizes some loss ℓ on the data
  - with *r(h)* the model size and λ the weight of this model size

$$\underset{h \in \mathcal{H}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \ell(h(x_i), y_i) + \lambda r(h)$$

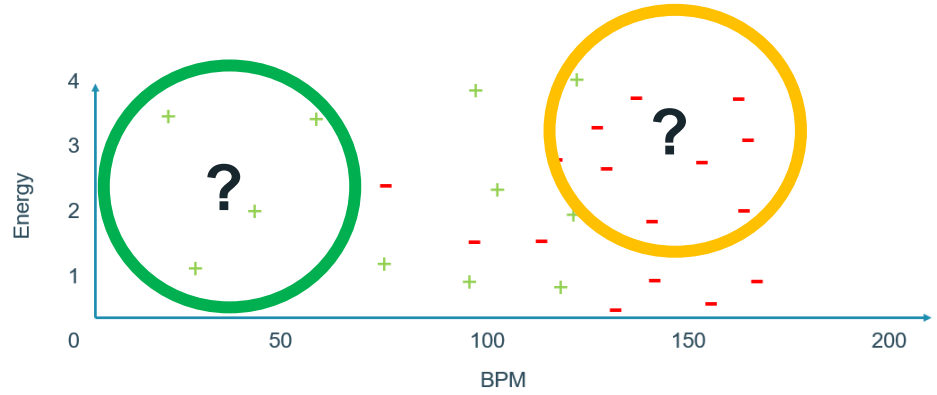
Lazy vs Eager learners

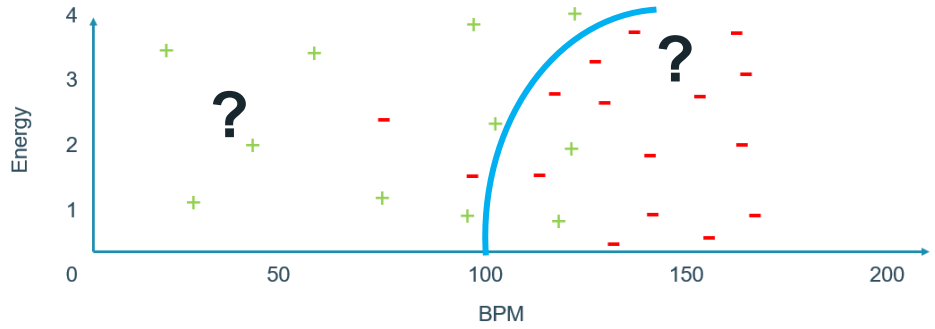
- Lazy learners: there is no explicit model
  - A new model is built on demand per query
  - Pushes all the work to prediction time



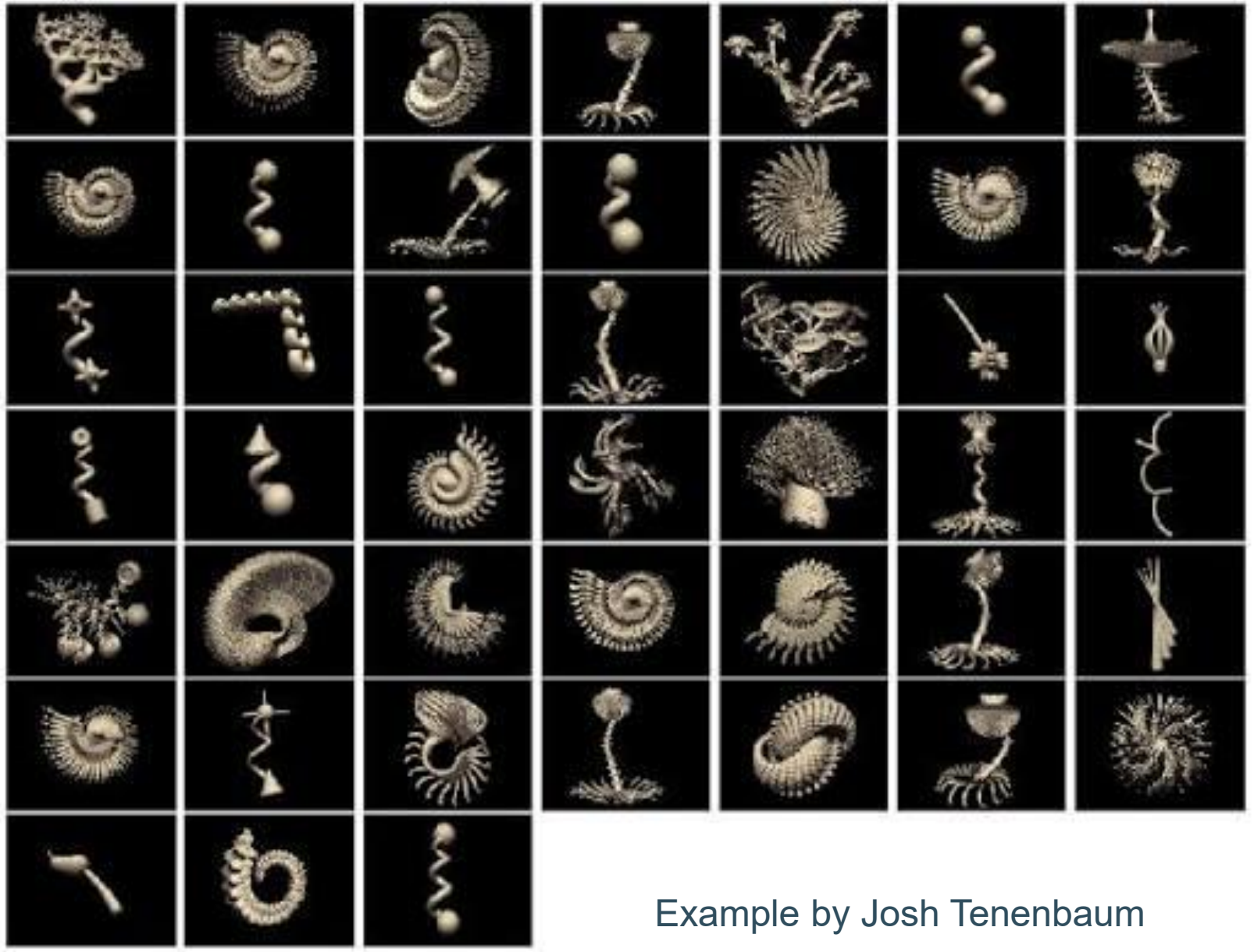
Lazy vs Eager learners

- Lazy learners: there is no explicit model
  - A new model is built on demand per query
  - Pushes all the work to prediction time
- Eager learners: Build explicit model
  - Build a global model that works for all examples
  - Work done at training time





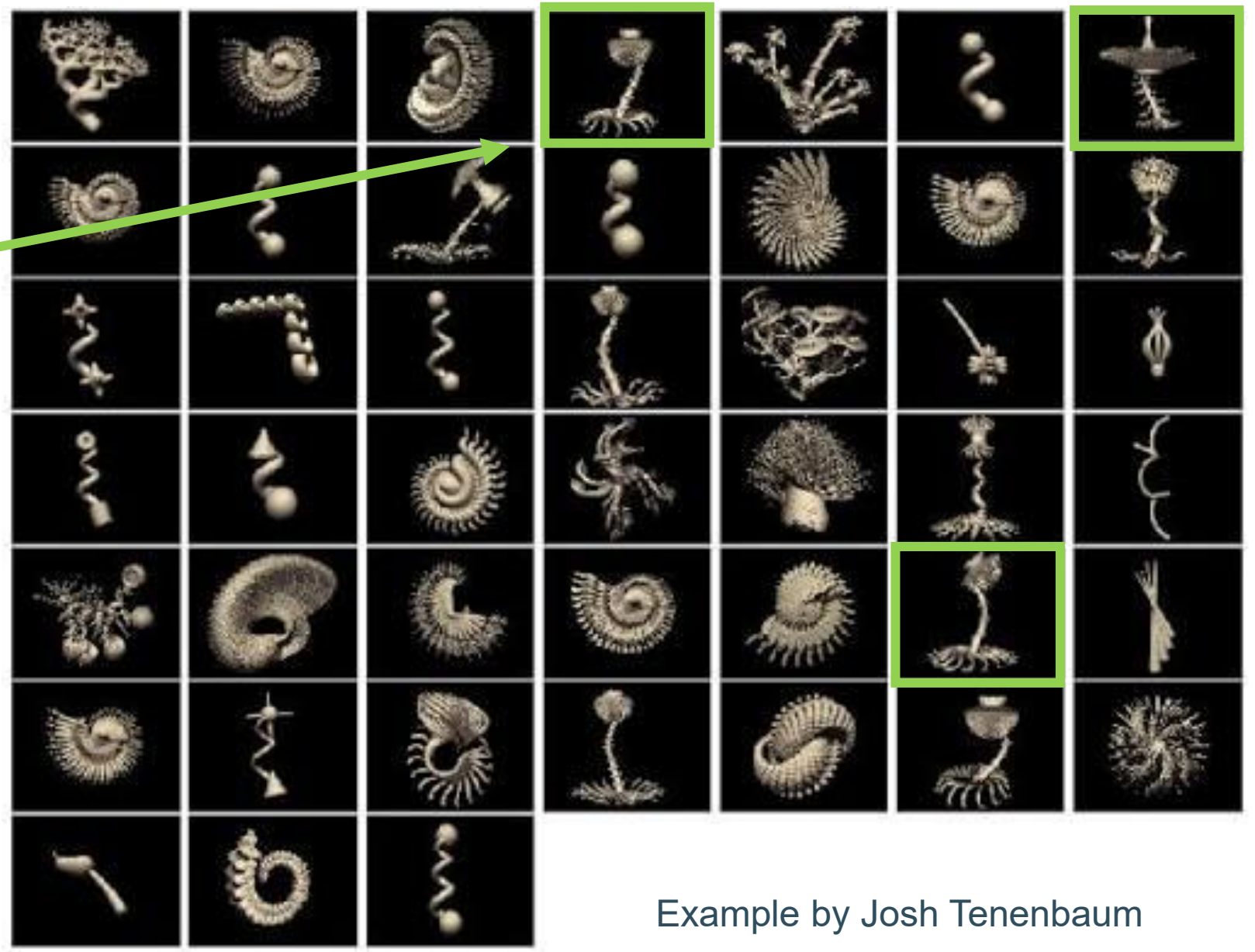
A real example of learning a model: k nearest neighbors (kNN)



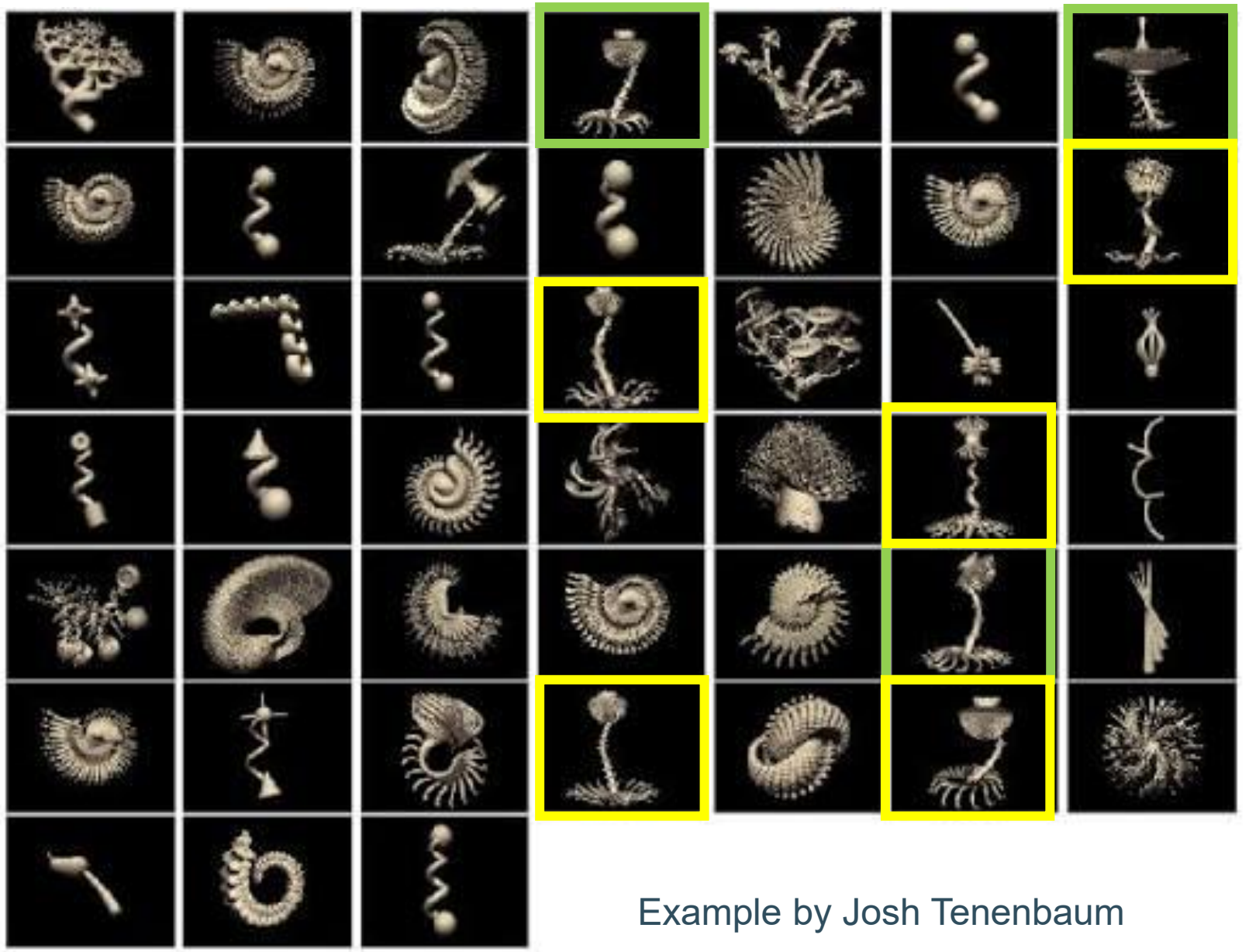
Example by Josh Tenenbaum

Tufa

Can you pick
Out the other
Tufa's?



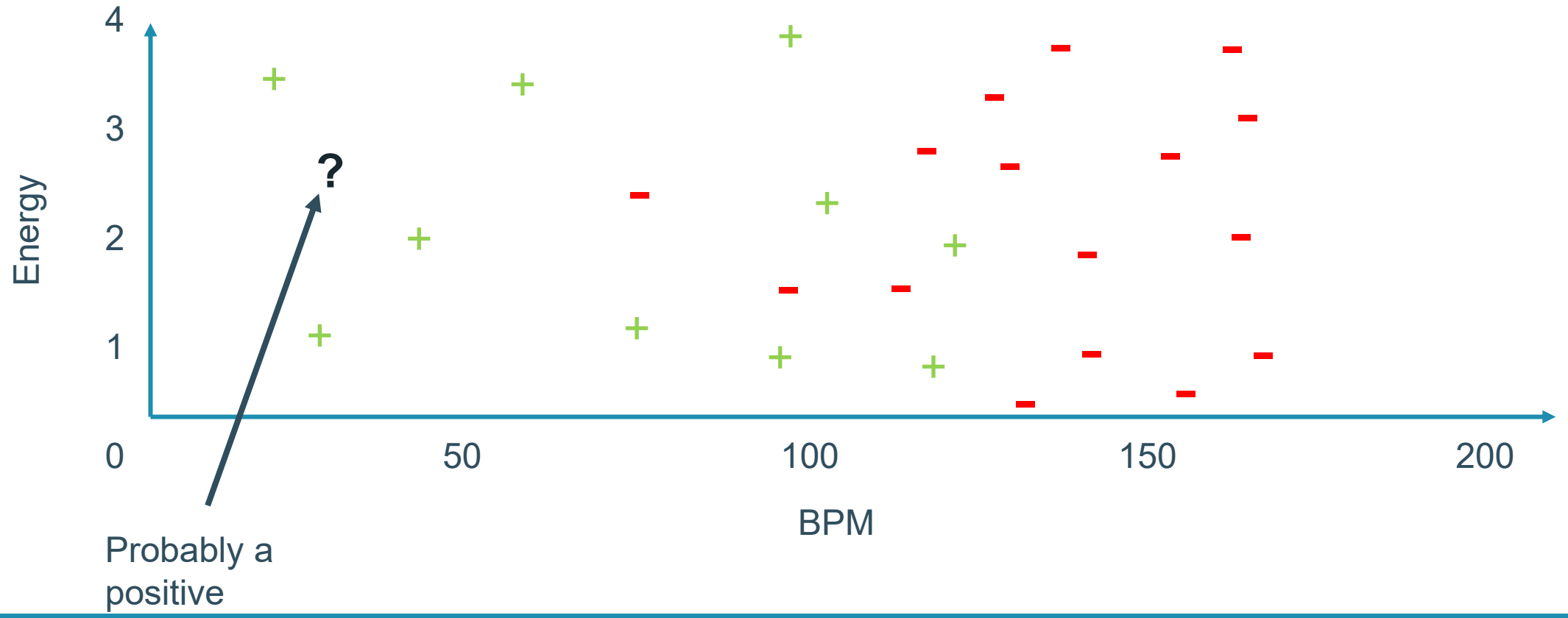
Example by Josh Tenenbaum



Example by Josh Tenenbaum

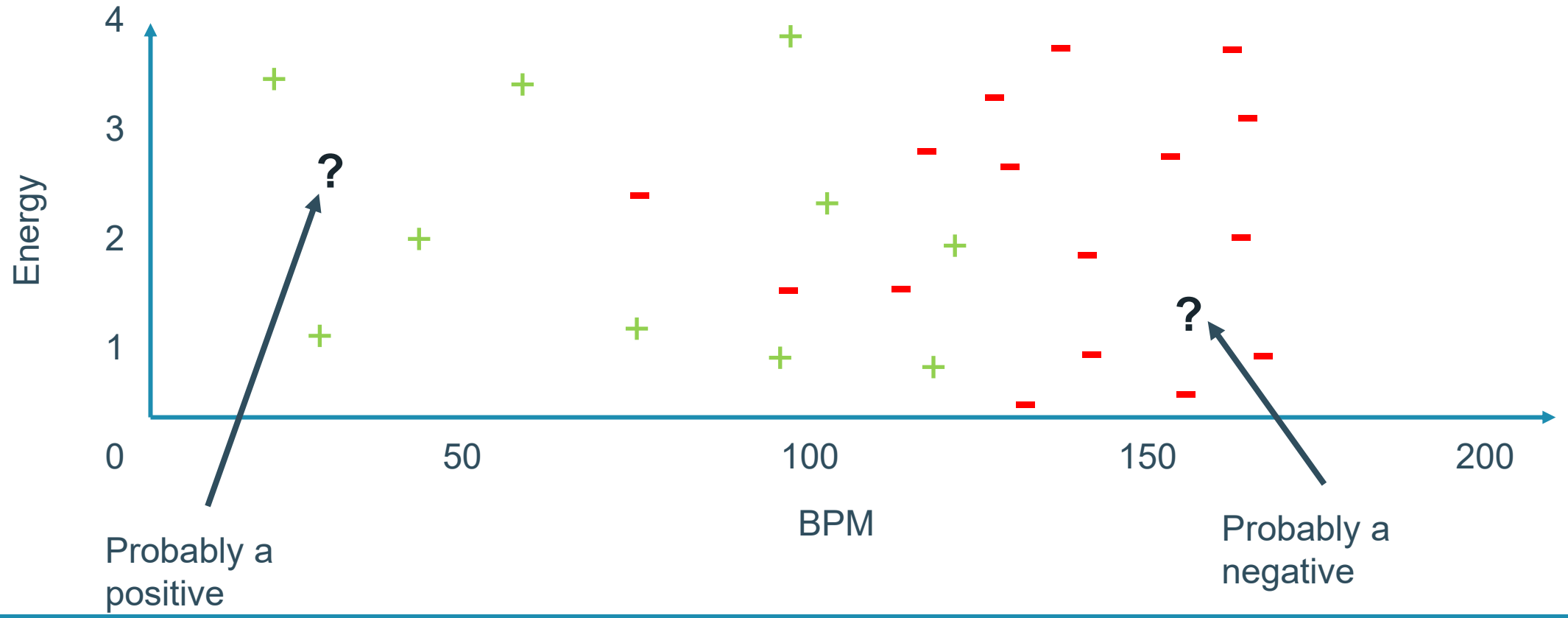
Instance-based learning intuitively

- If it walks like a duck and quacks like a duck, then it's probably a duck



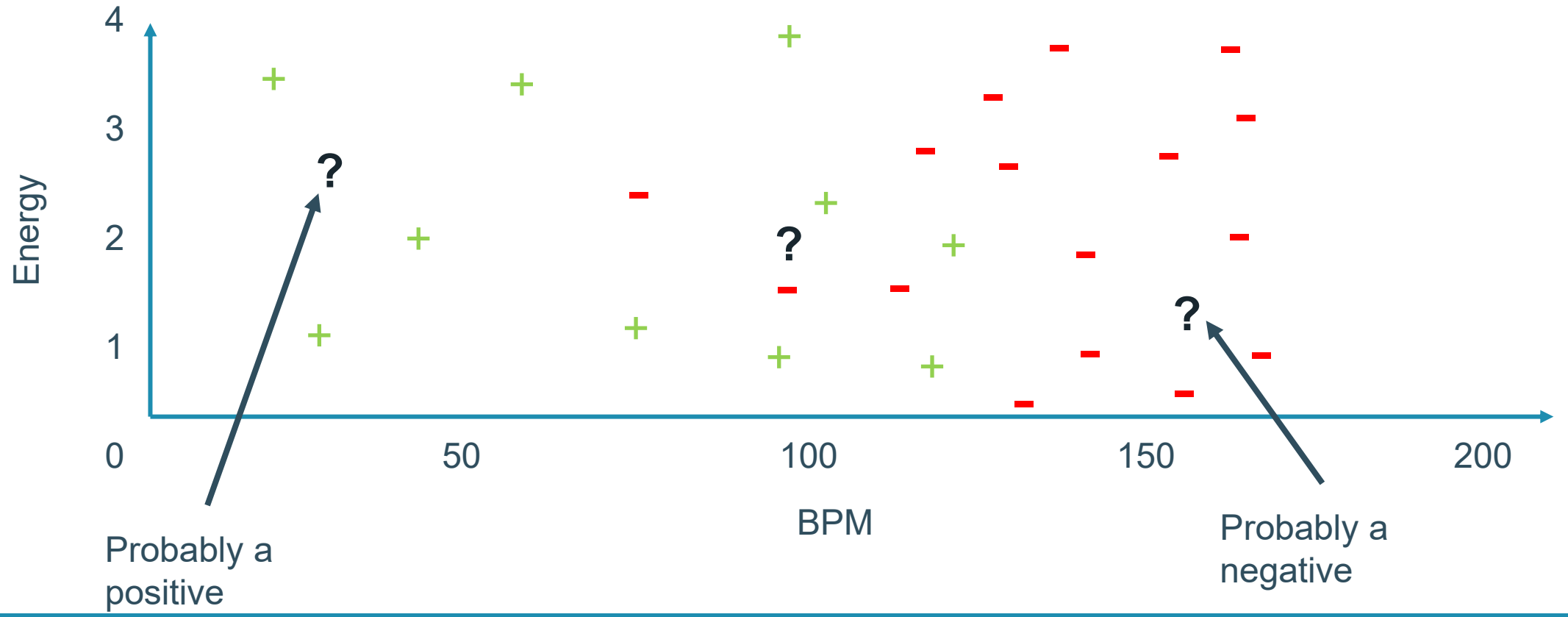
Instance-based learning intuitively

- If it walks like a duck and quacks like a duck, then it's probably a duck



Instance-based learning intuitively

- If it walks like a duck and quacks like a duck, then it's probably a duck



Even though it seems trivial, why should you care about nearest neighbors?

- Recommender systems
- Medical case comparison
- Web and image search
- News articles
- Plagiarism detection
- Weather forecasting
- Retrieval augmented generation (RAG)

First attempt: Memorize the data

Let $T=\{(x_i, y_i)\}$ where $i \in \{1,..,n\}$

Classify example e

if there exists $x_i \in T$ such that $e_1==x_{i,1} \wedge e_2==x_{i,2} \wedge \cdots \wedge e_d==x_{i,d}$ → Return y_i

else return ?

First attempt: Memorize the data

Let $T=\{(x_i, y_i)\}$ where $i \in \{1,..,n\}$

Classify example e

if there exists $x_i \in T$ such that $e_1==x_{i,1} \wedge e_2==x_{i,2} \wedge \cdots \wedge e_d==x_{i,d}$ → Return y_i

else return ?

Problems:

- Cannot generalize beyond the training data
- Exact match is very restrictive

K Nearest Neighbors (kNN)

- Find the *k* nearest labelled examples and return their combined decision

Let T = $\{(x_j, y_j)\}$ where $_j \in \{1,\ldots,n\}$

Classify Example *e*

priority queue *pq* of size k

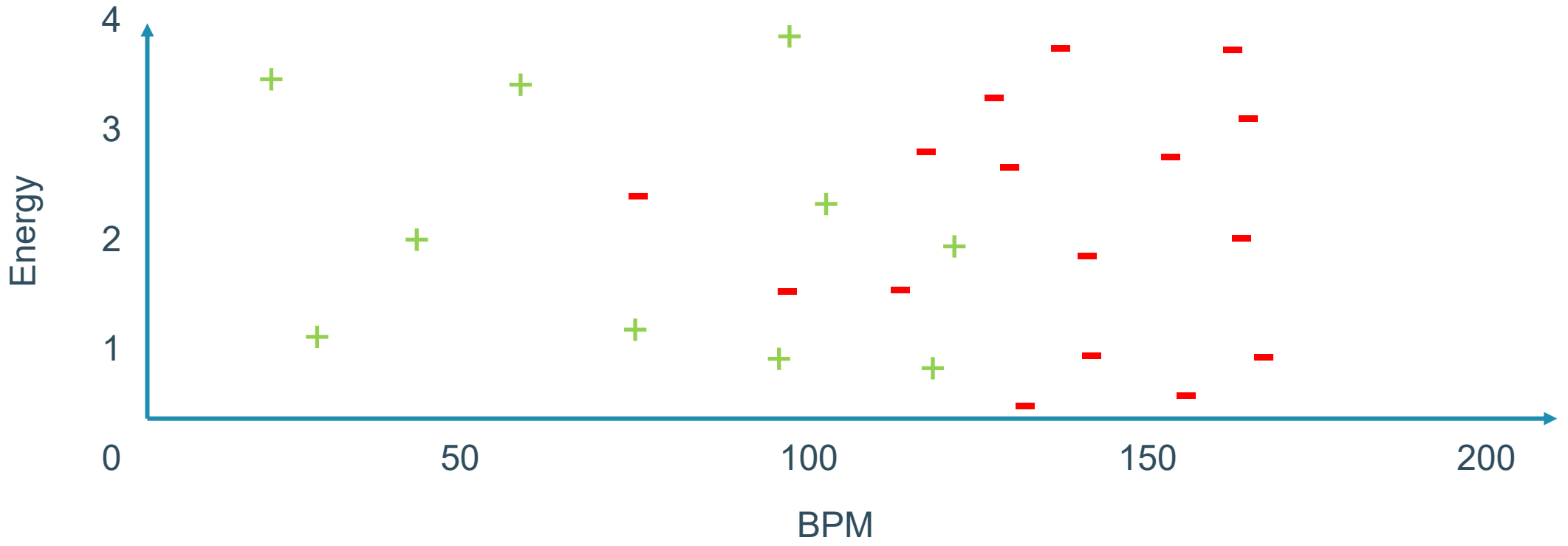
forall $(x_j, y_j) \in$ T

if (dist(e, x_j) < pq.max) then pq.enqueue(x_j ,y_j)

prediction = combine($y_j \in$ pq)

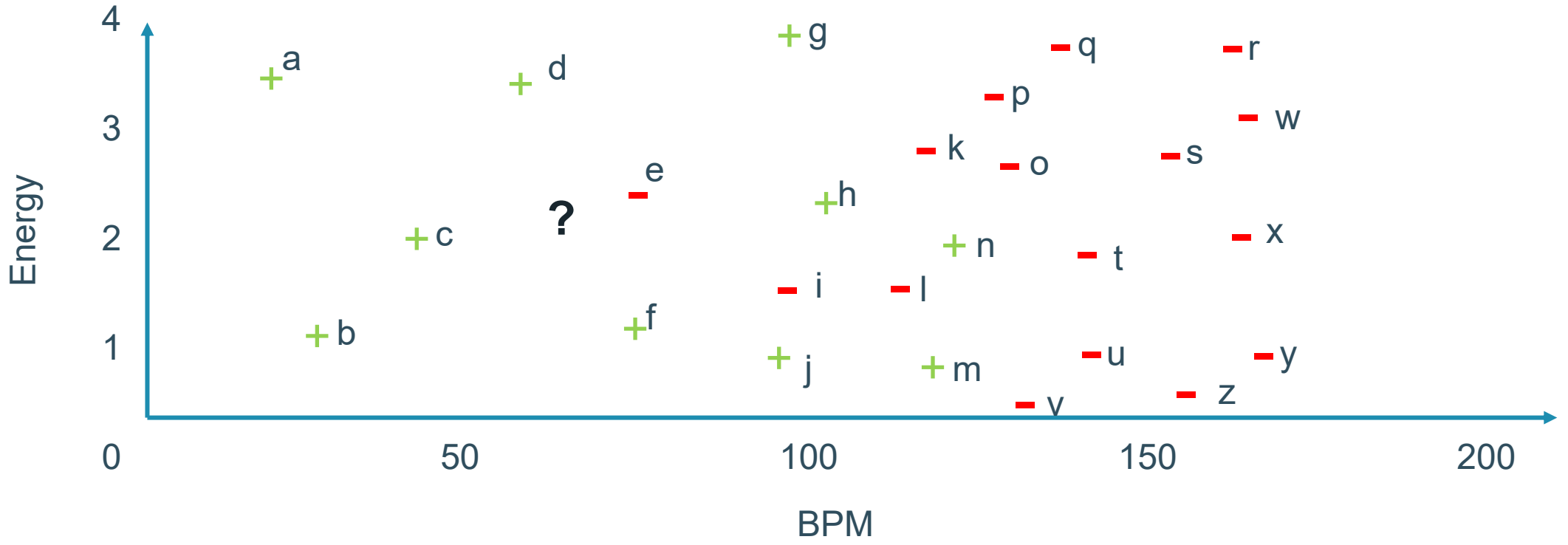
return prediction

Example of 1-NN



Example of 1-NN

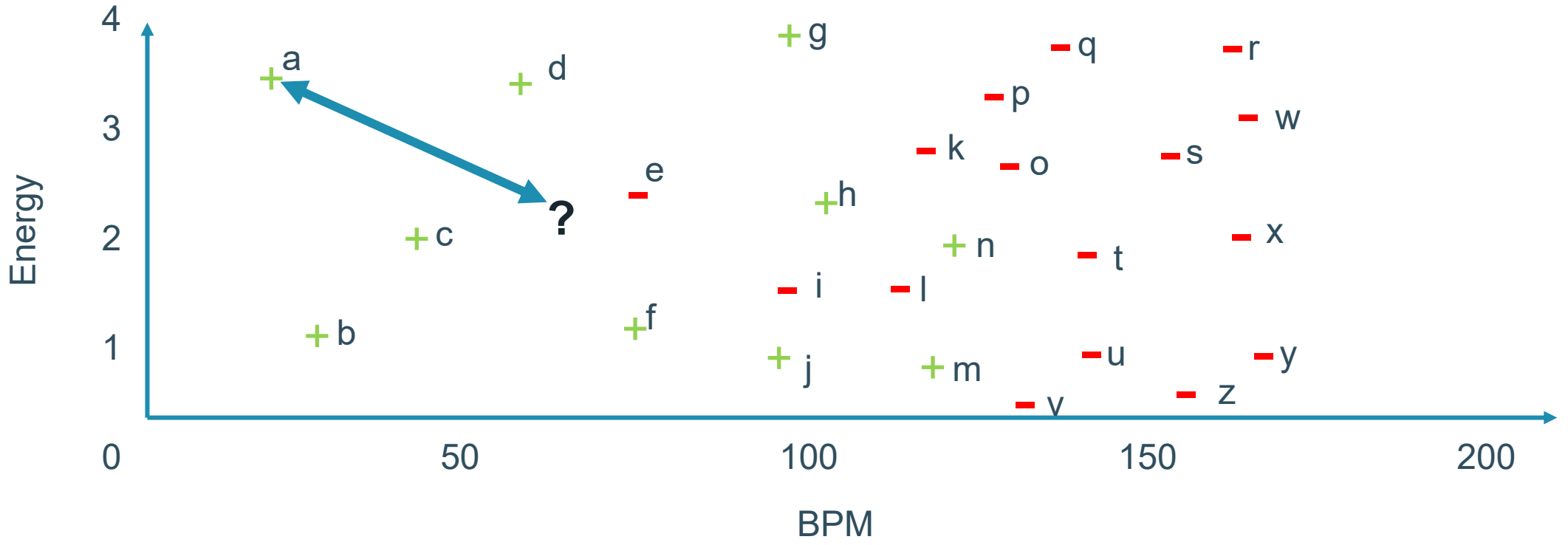
Queue



Example of 1-NN

Queue

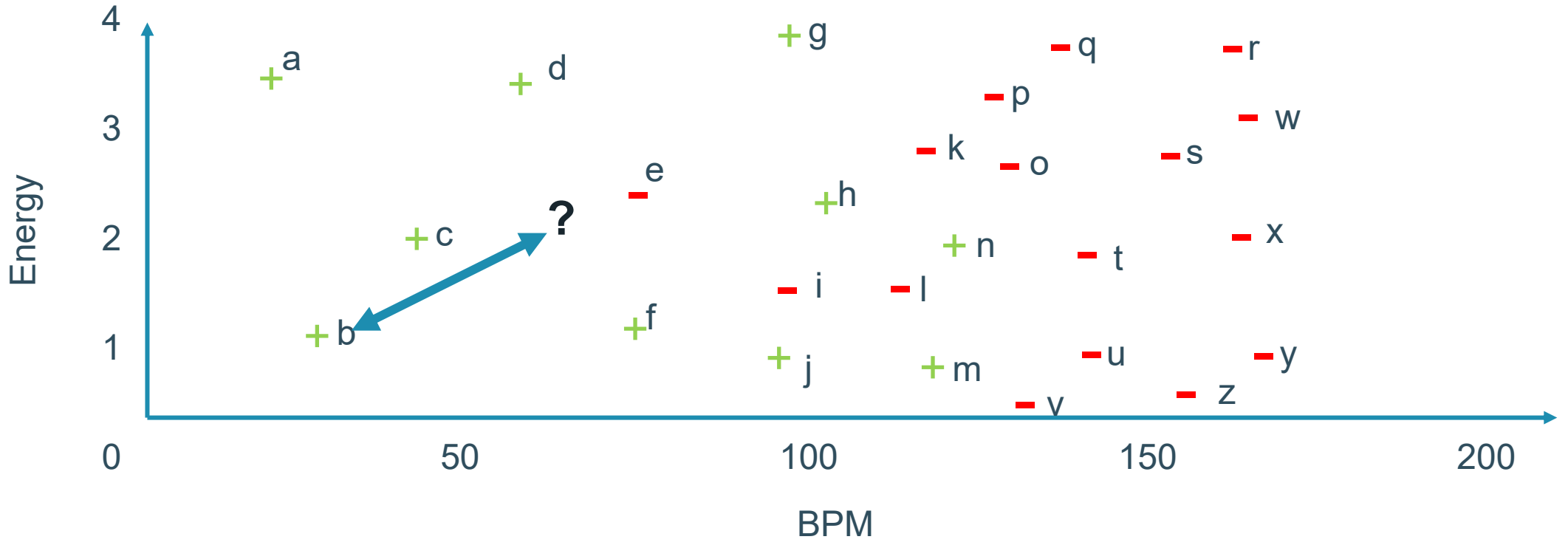
Example	Dist
a	10



Example of 1-NN

Queue

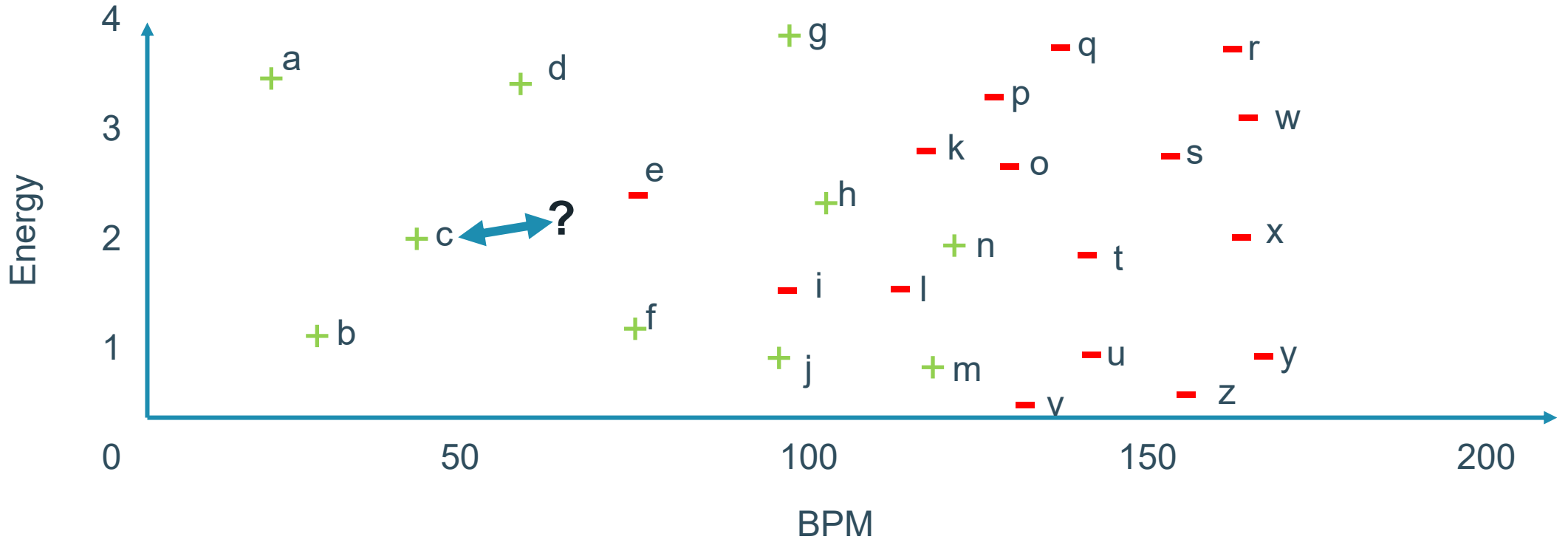
Example	Dist
b	8



Example of 1-NN

Queue

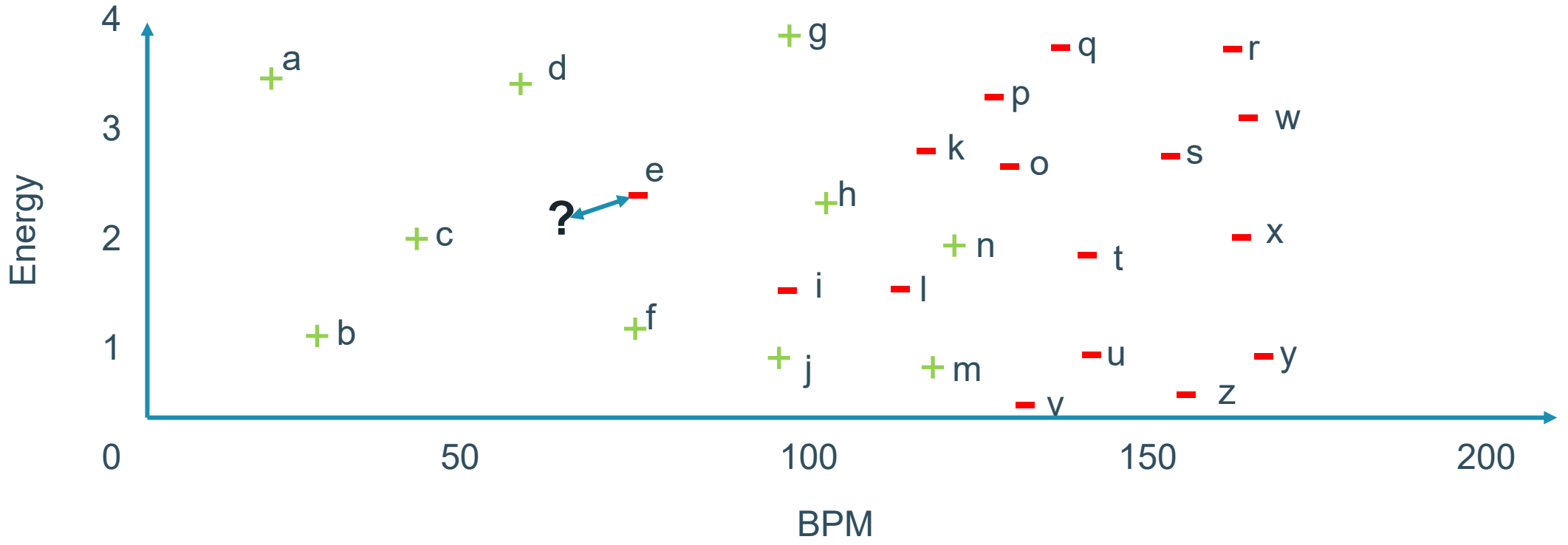
Example	Dist
c	3



Example of 1-NN

Queue

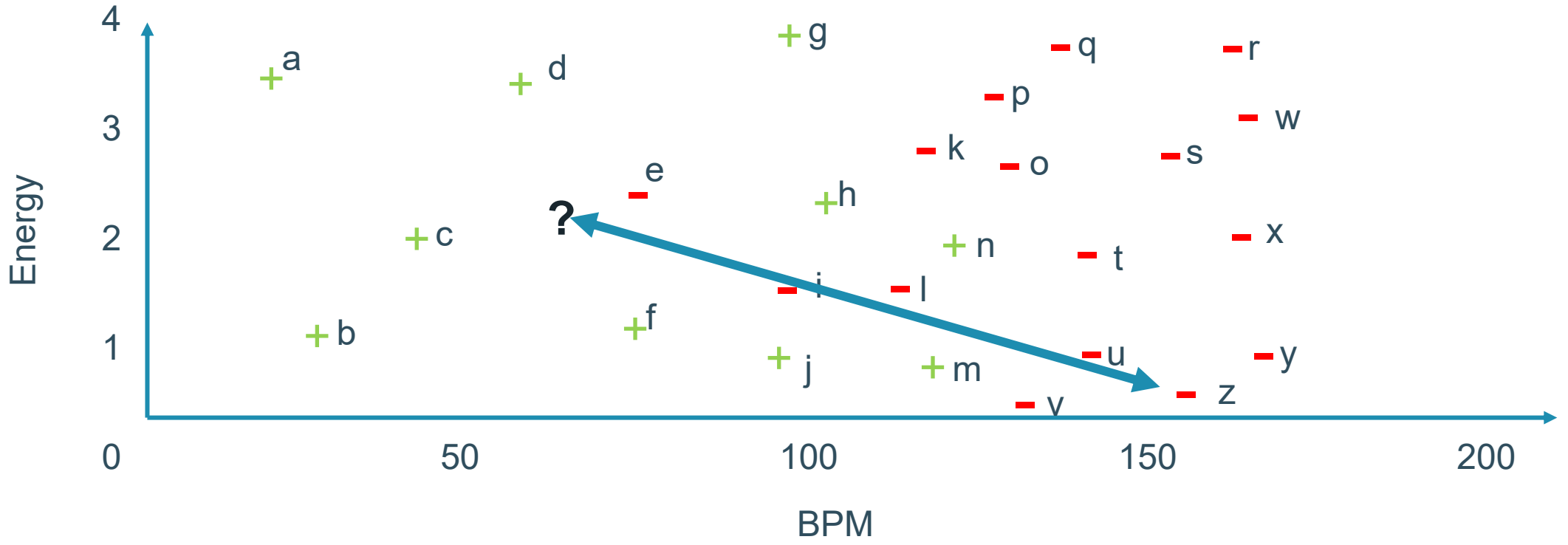
Example	Dist
e	2



Example of 1-NN

Queue

Example	Dist
e	2



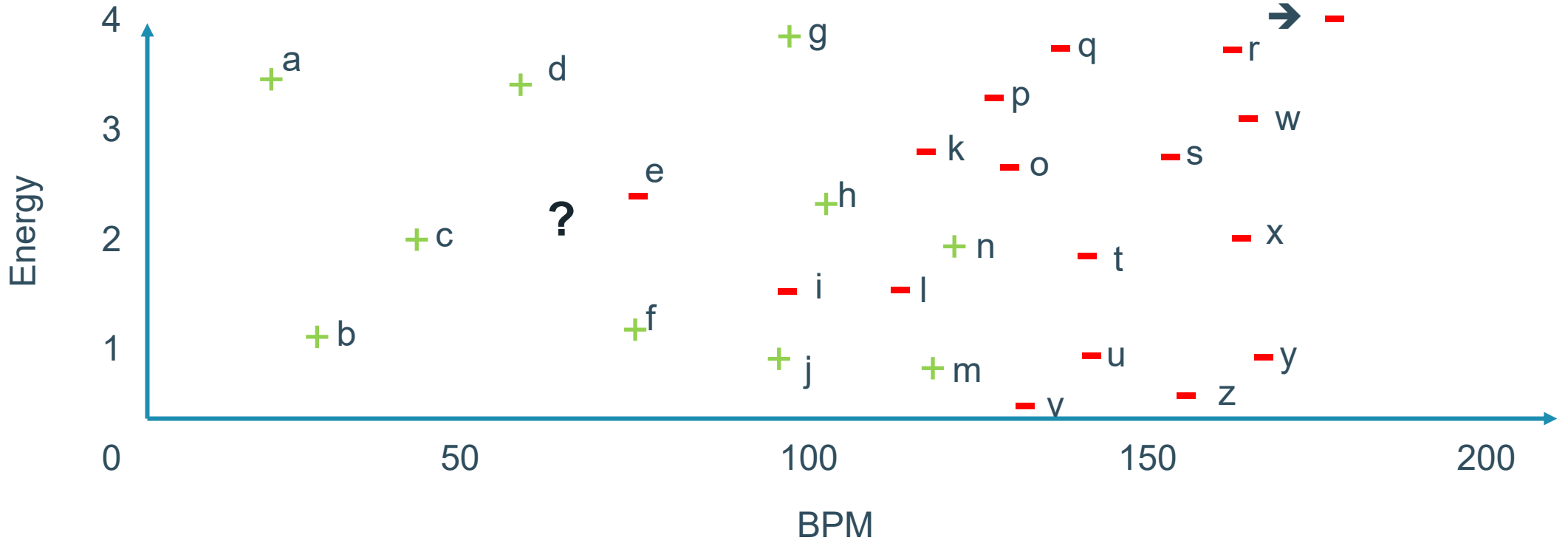
Example of 1-NN

Queue

Example	Dist
e	2

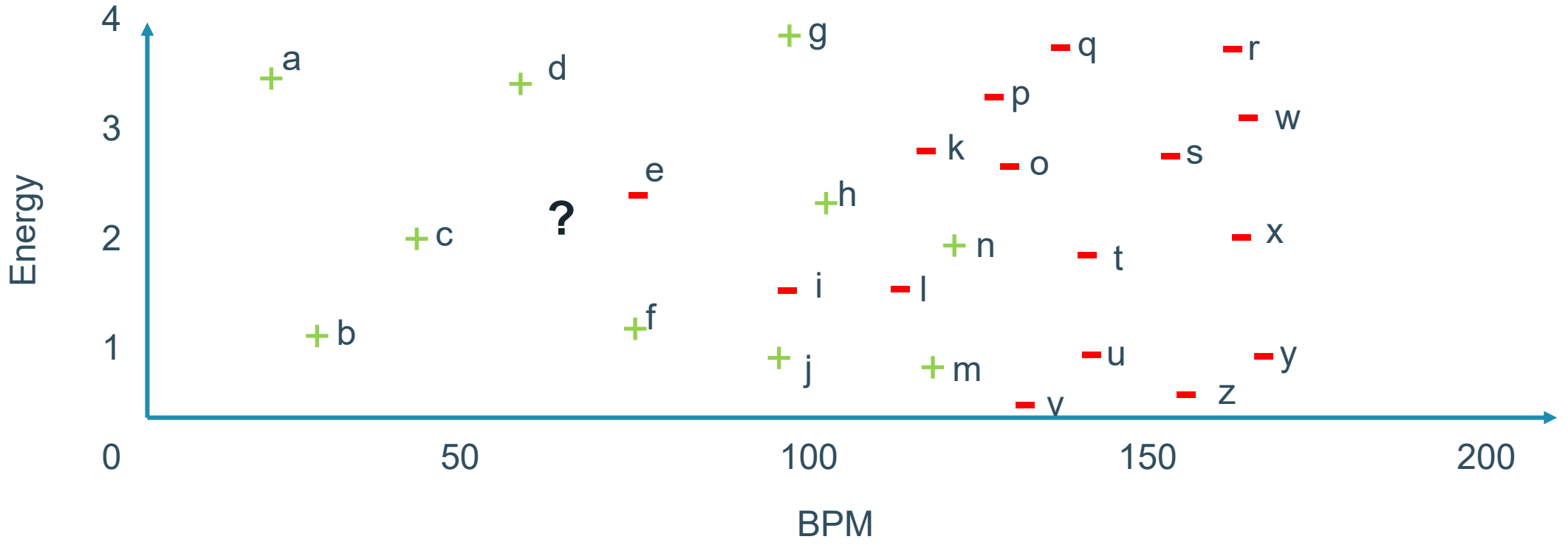
➔ Take the label of ‘e’

➔ −



Example of 3-NN

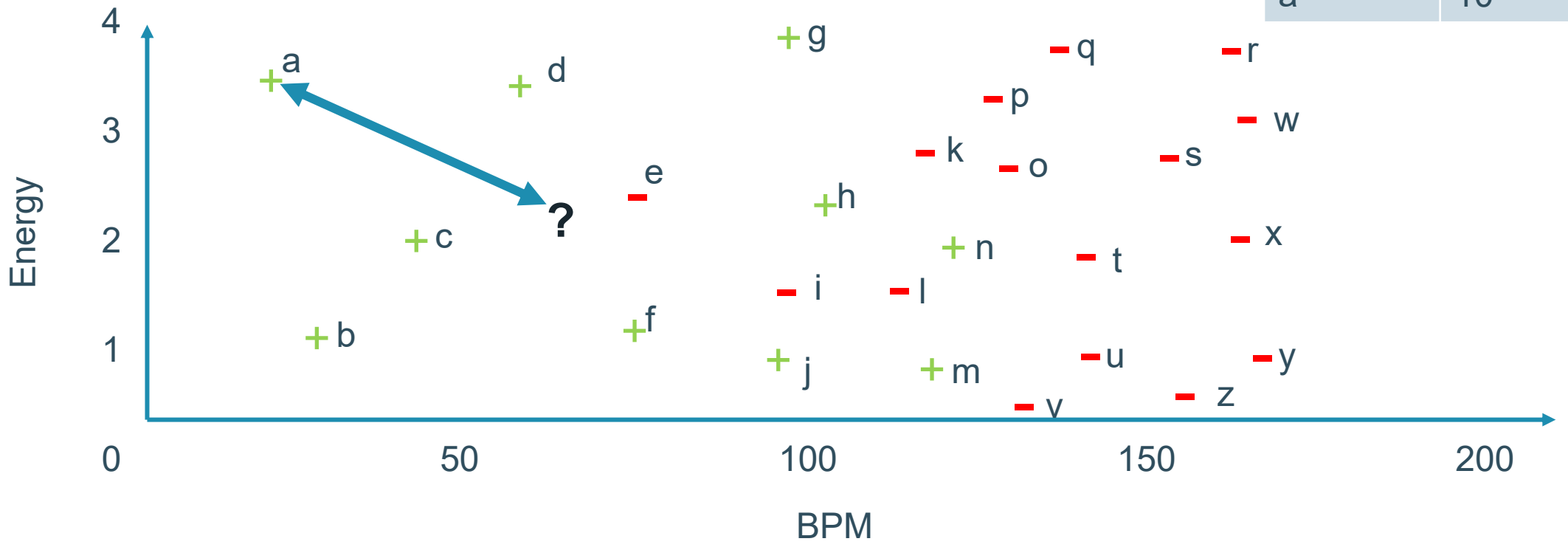
Queue



Example of 3-NN

Queue

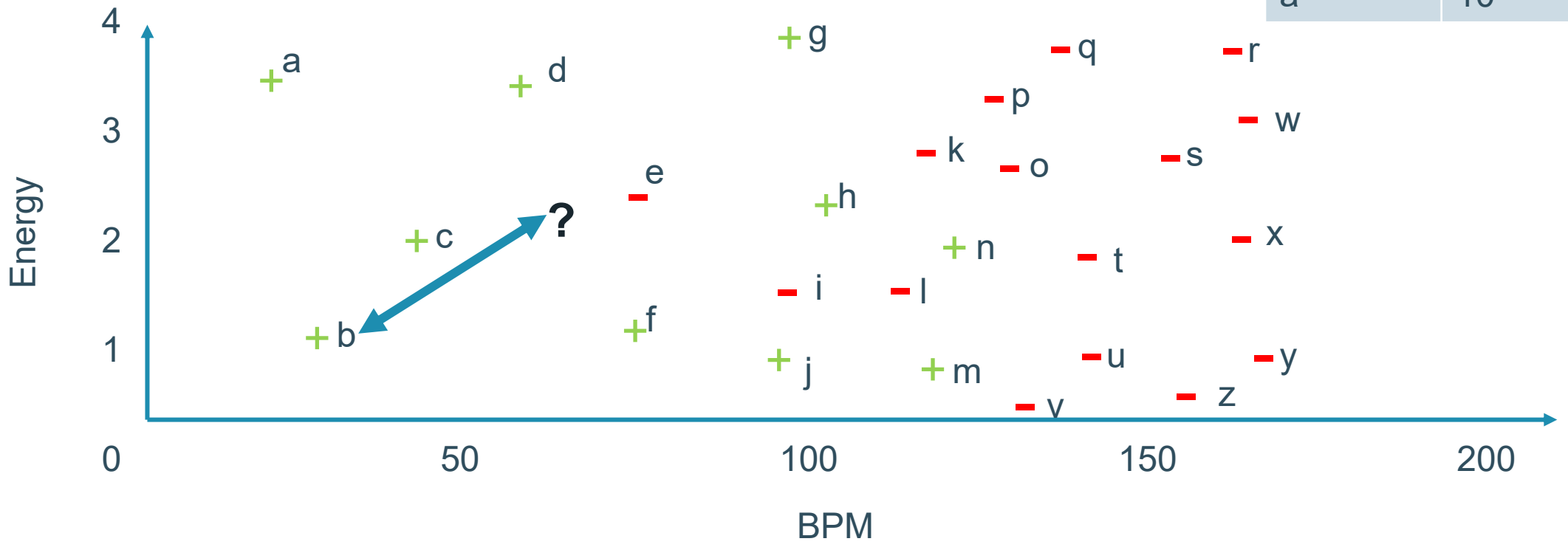
Example	Dist
a	10



Example of 3-NN

Queue

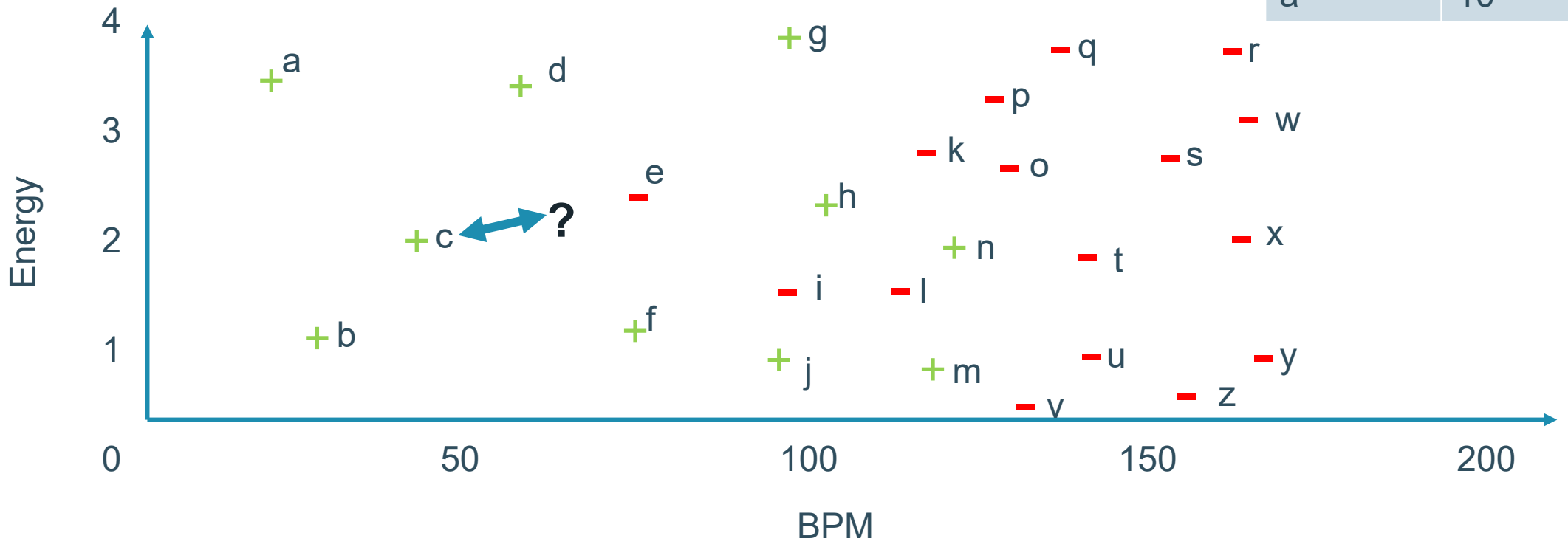
Example	Dist
b	8
a	10



Example of 3-NN

Queue

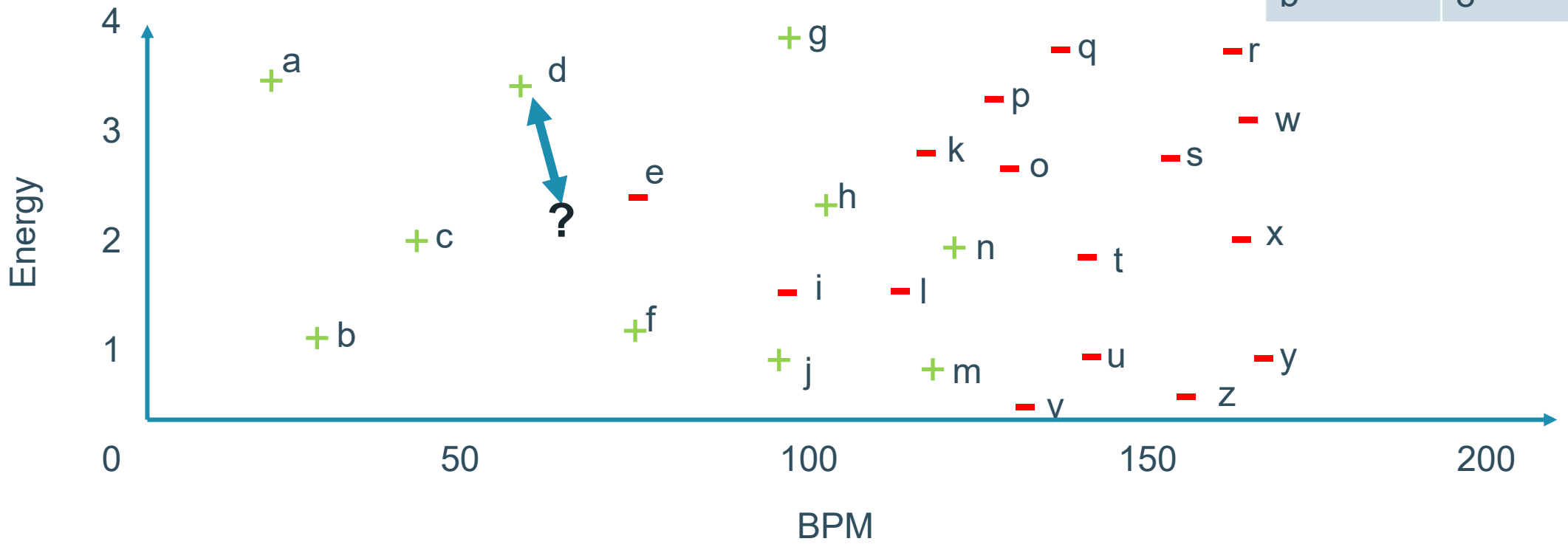
Example	Dist
c	3
b	8
a	10



Example of 3-NN

Queue

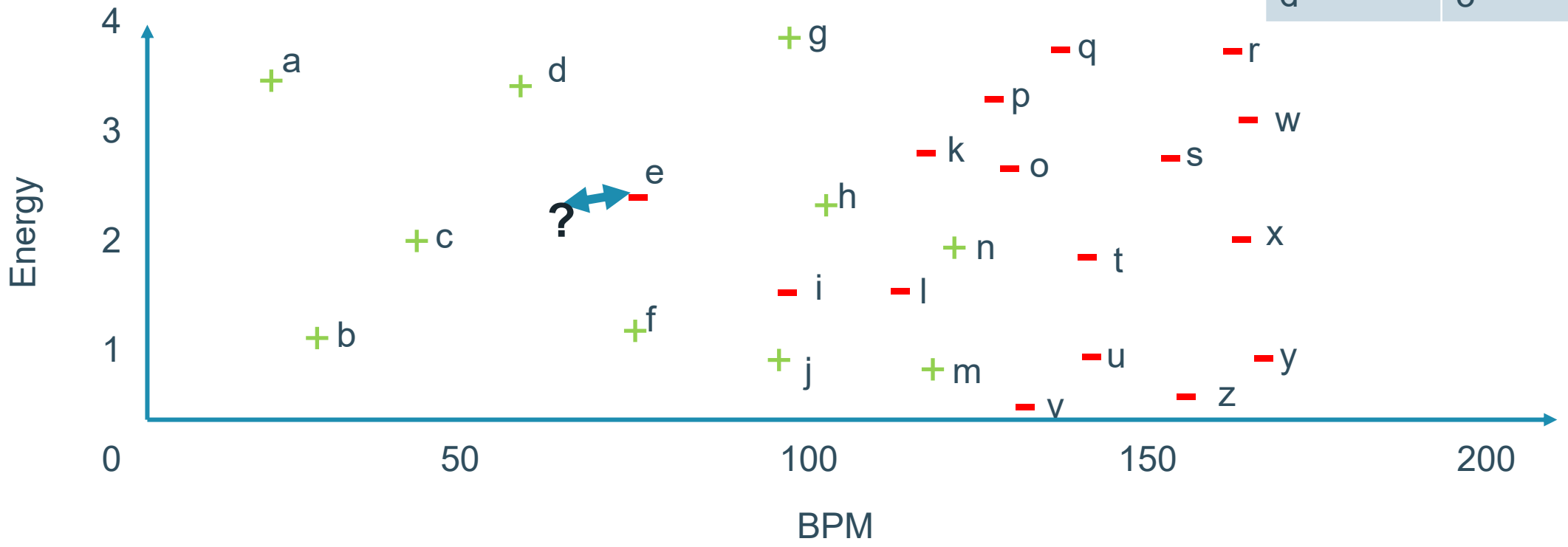
Example	Dist
c	3
d	3
b	8



Example of 3-NN

Queue

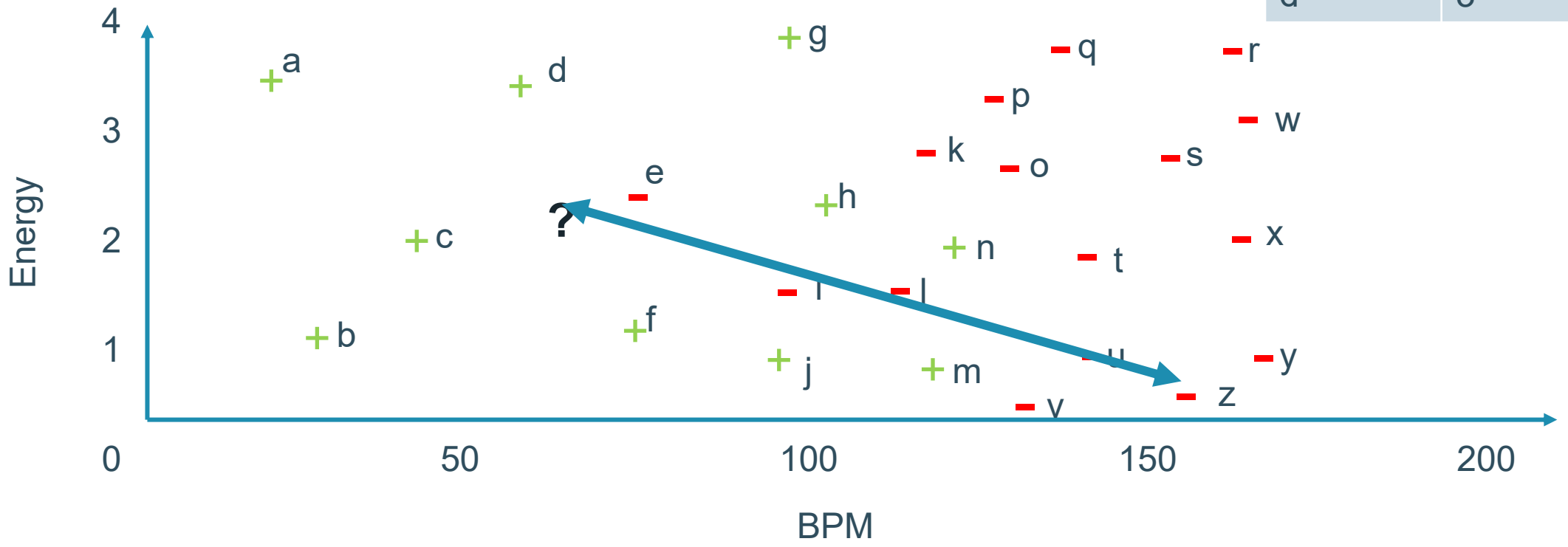
Example	Dist
e	2
c	3
d	3



Example of 3-NN

Queue

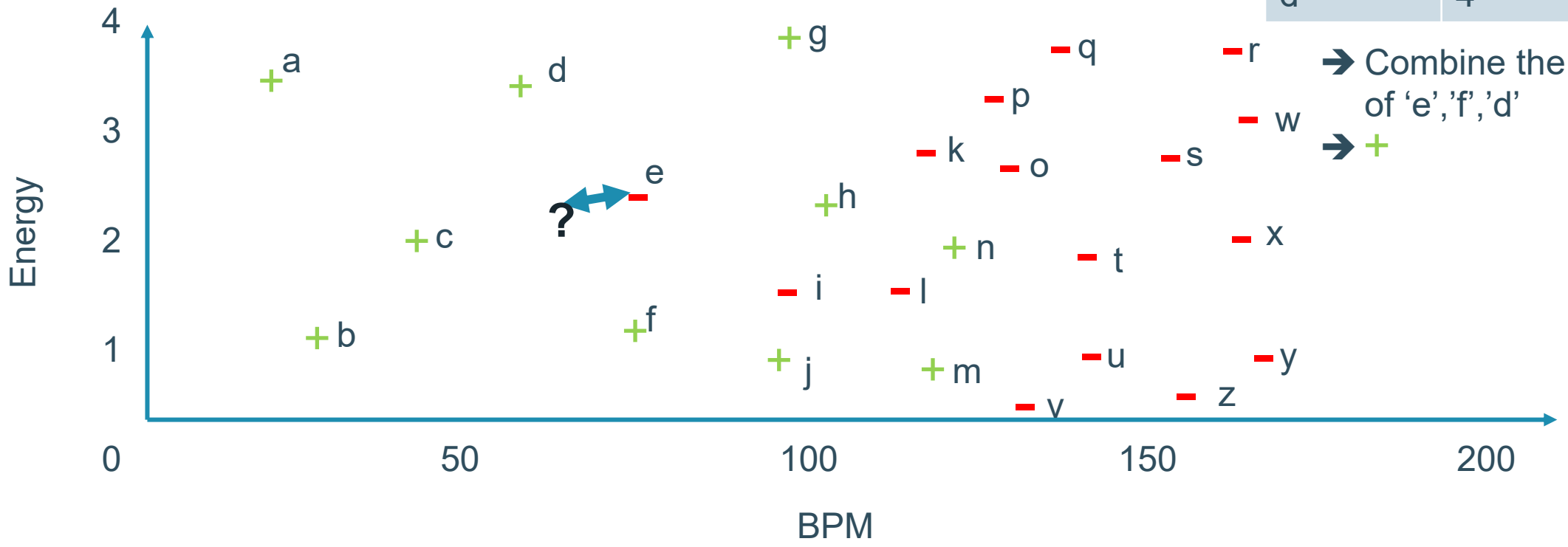
Example	Dist
e	2
f	3
d	3



Example of 3-NN

Queue

Example	Dist
e	2
c	3
d	4



➔ Combine the labels of ‘e’,’f’,’d’

➔ +

K Nearest Neighbors (kNN)

- Find the *k* nearest labelled examples and return their combined decision

1. What distance functions are possible?
2. How expressive is kNN?
3. How can we combine a test example's neighbors to make a prediction?
4. What should we worry about when using kNN?

What distances are possible?

Similarities and distances

- **Similarities:** Higher number = more similar → neighbors are most similar
- **Distances:** Higher number = farther apart → Neighbors are closest

  Constraints:

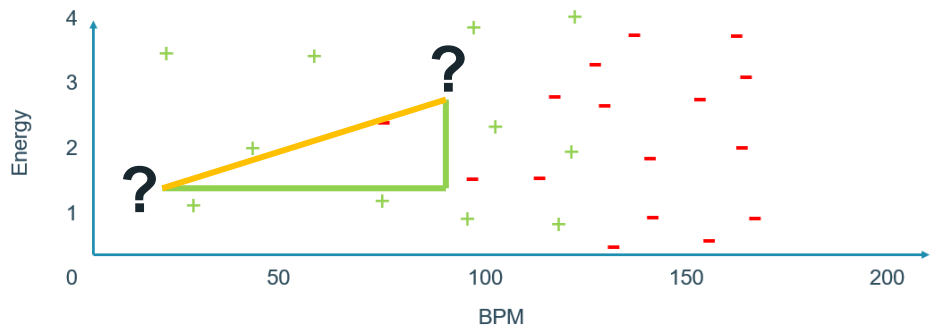
  1. Dist(i,j)≥0
  2. Dist(i,i) = 0
  3. Dist(i,j) = Dist(j,i)
  4. Dist(i,j) ≤ Dist(i,k) + Dist(k,j)

- Easy to convert distance to similarity, but not vice versa

$$sim(i,j) = \frac{1}{1 + Dist(i,j)}$$

Classic distance measures

- All discrete features: **Hamming distance** counts the number of features examples differ on $Hamming(x_i, x_j) = \Sigma_{k=1}^{d} \mathbb{I}(x_{i,k} \neq x_{j,k})$
- All real-valued features
  - **Manhattan** $(x_i, x_j) = \Sigma_{k=1}^{d} |x_{i,k} - x_{j,k}|$
  - **Euclidean** $(x_i, x_j) = \sqrt{\Sigma_{\mathrm{k=1}}^{\mathrm{d}} (x_{i,k} - x_{j,k})^2}$



More advanced measures

- **Value difference metric:** attribute values are close if they make similar predictions

$$\mathrm{vdm}(x_{i.k}, x_{j,k}) = \sum_{c=1}^{|Y|} |P(y_c|x_{i,k}) - P(y_c|x_{j,k})|$$

- Mixed continuous and discrete attributes

$$\mathrm{dist}(x_i, x_j) = \sum_{k=1}^{d} \delta(x_{i,k}, x_{j,k})$$

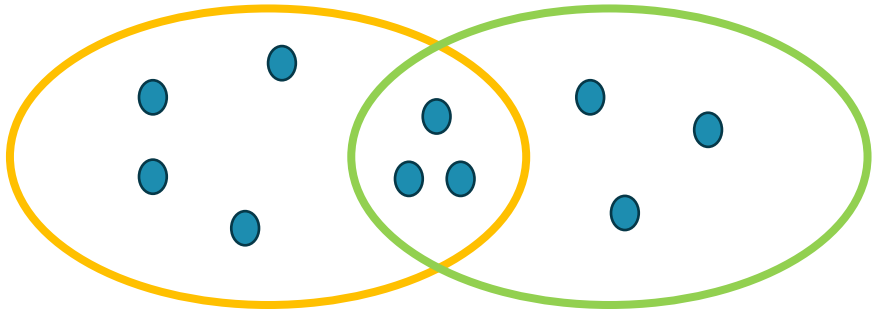
$$\delta(x_{i,k}, x_{j,k}) = \begin{cases} |x_{i,k} - x_{j,k}| & \text{If } x_{*,k} \text{ is continuous} \\ 1 - \mathbb{I}(x_{i.k} = x_{j,k}) & \text{If } x_{*,k} \text{ is discrete} \end{cases}$$

Jaccard: Set comparisons

- Jaccard similarity: $\mathrm{sim}(S_i, S_j) = \frac{S_i \cap S_j}{S_i \cup S_j}$

- Jaccard distance: $1 - \mathrm{sim}(S_i, S_j)$

- Example
  - Sim(S_1,S_2)=0.3
  - Dist(S_1,S_2)=0.7



Structured objects

- **Edit distance** for strings: the number of required edits to transform a string into another string
  - Edit_distance(learning, leaning)=1 (remove ‘r’)
  - Edit_distance(learning, carving)=3 (remove ‘l’, replace ‘e’ with ‘c’, replace ‘n’ with ‘v’)

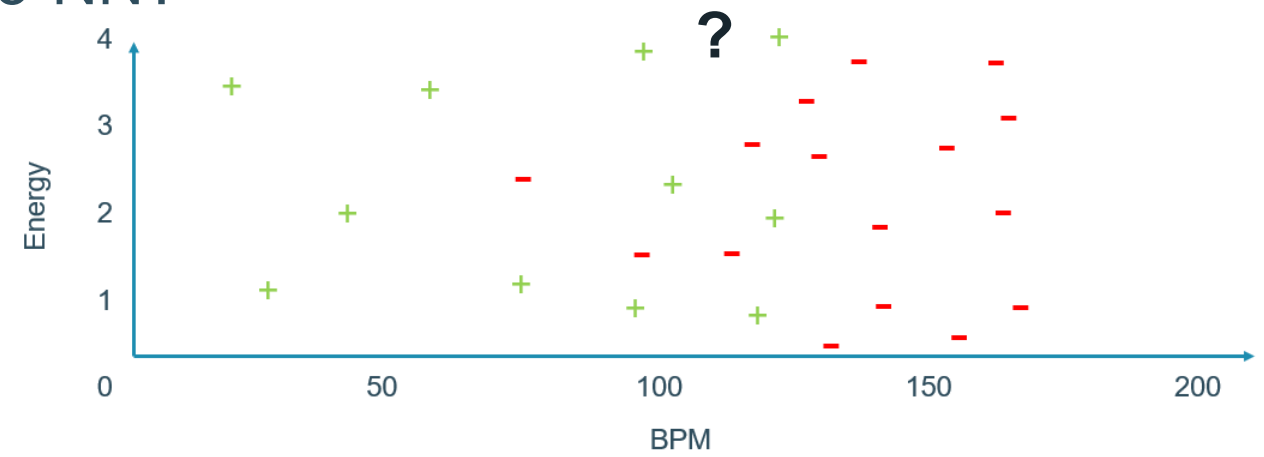
- **Cosine similarity** of two vectors

$$\mathrm{sim}(x_i, x_j) = \frac{\sum_{k=1}^{d} x_{i,k} x_{j,k}}{\sqrt{\sum_{k=1}^{d} x_{i,k}^2} \sqrt{\sum_{k=1}^{d} x_{j,k}^2}}$$

How can we combine neighbors' labels?

Predicting classes from neighbors labels

- Majority vote
- What about this for 5-NN?



-Solution: distance weighted kNN

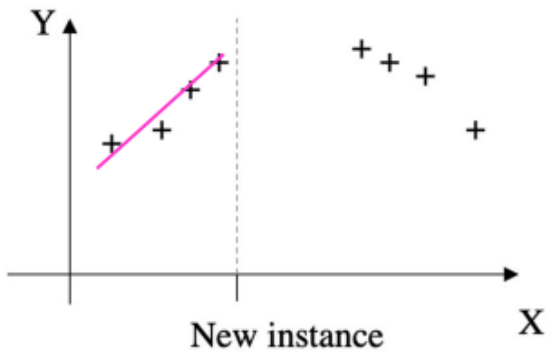
$$f(x_q) = argmax_y \Sigma_{i=1}^{k} w_i \mathbb{I}[y_i = y] \qquad w_i = \frac{1}{dist(x_q, x_i)^2}$$

Predicting real values with kNN

- Average of the nearest neighbors

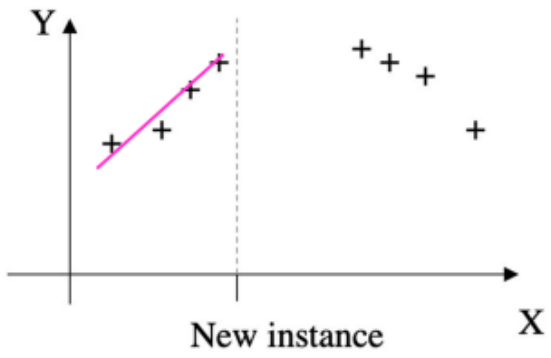
Predicting real values with kNN

- Average of the nearest neighbors
- What about this?



Predicting real values with kNN

- Average of the nearest neighbors
- What about this?



- Solution: Local regression model
  - Learn model on k nearest neighbors
  - Use the local model to make the prediction

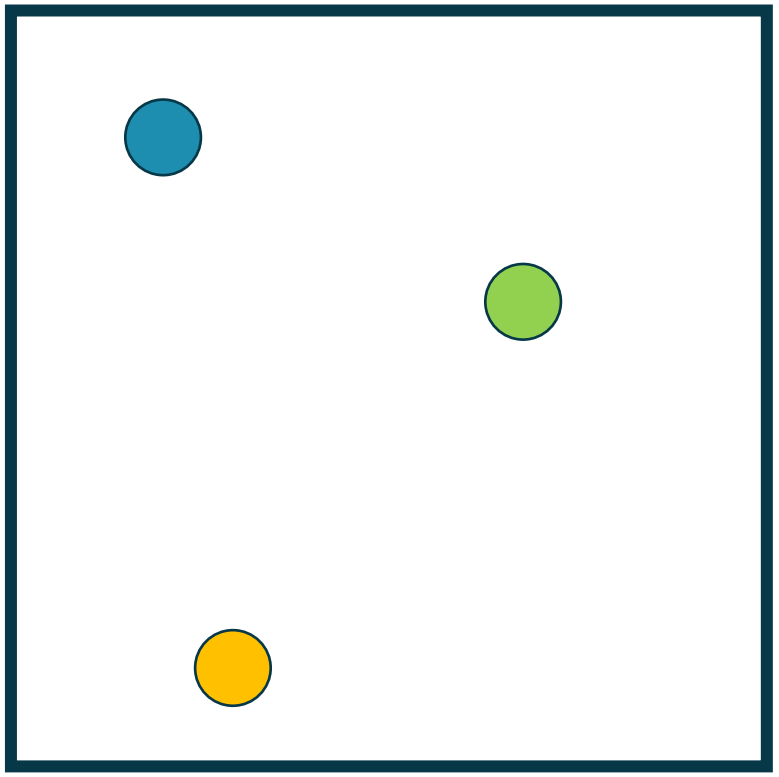
How expressive is kNN?

What do we mean by expressive?

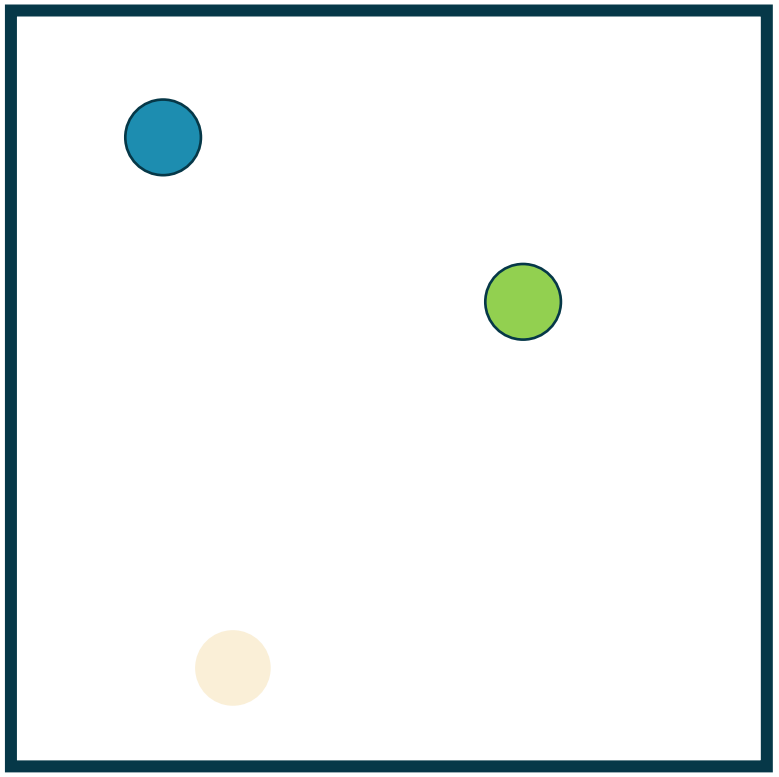
- Expressive = expressiveness of kNN's hypothesis space

- In other words
  - How complex are the concepts that kNN can represent?
  - How does kNN partition the feature space?

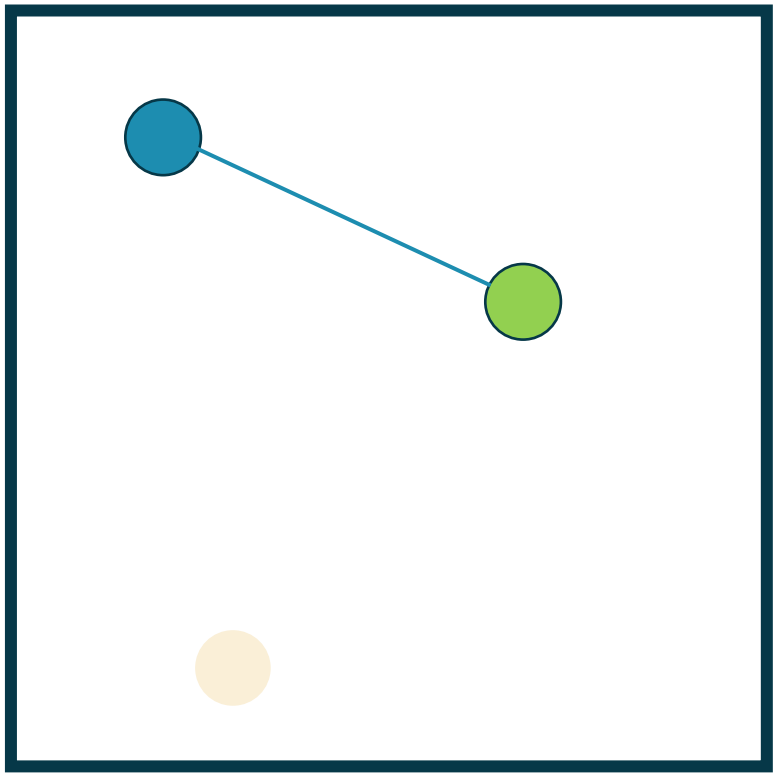
Let’s look at this geometrically



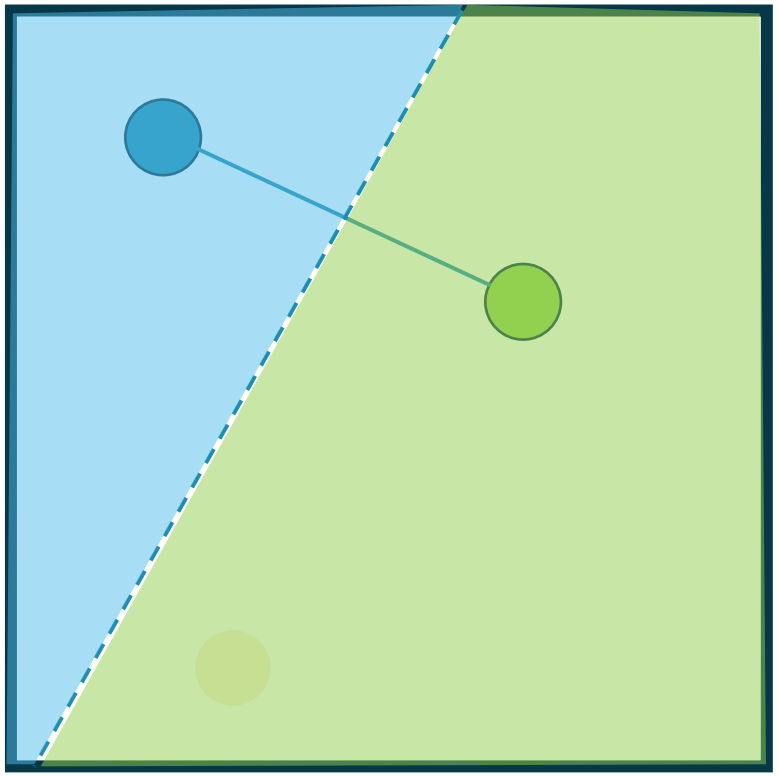
Let’s look at this geometrically



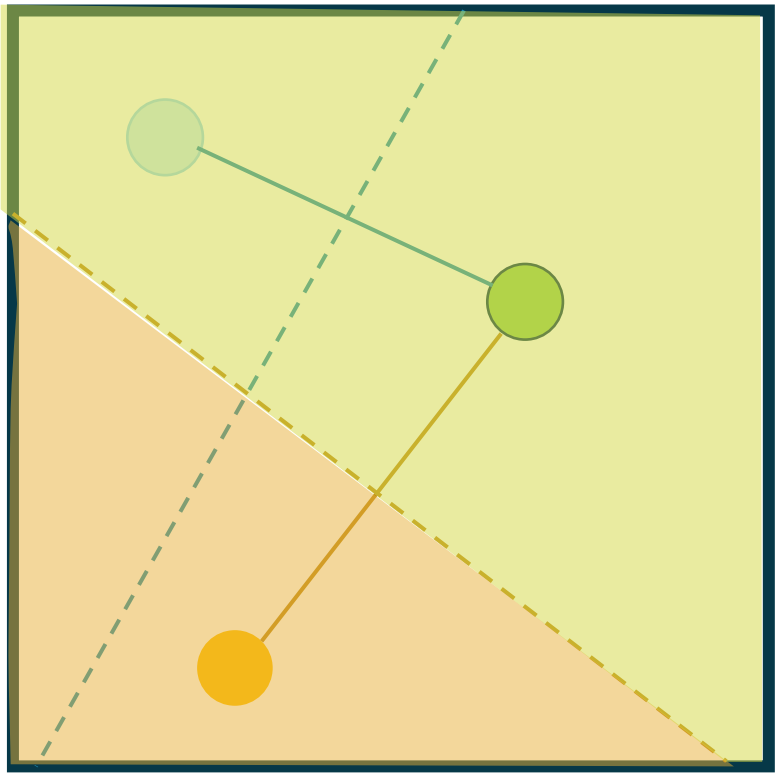
Let’s look at this geometrically



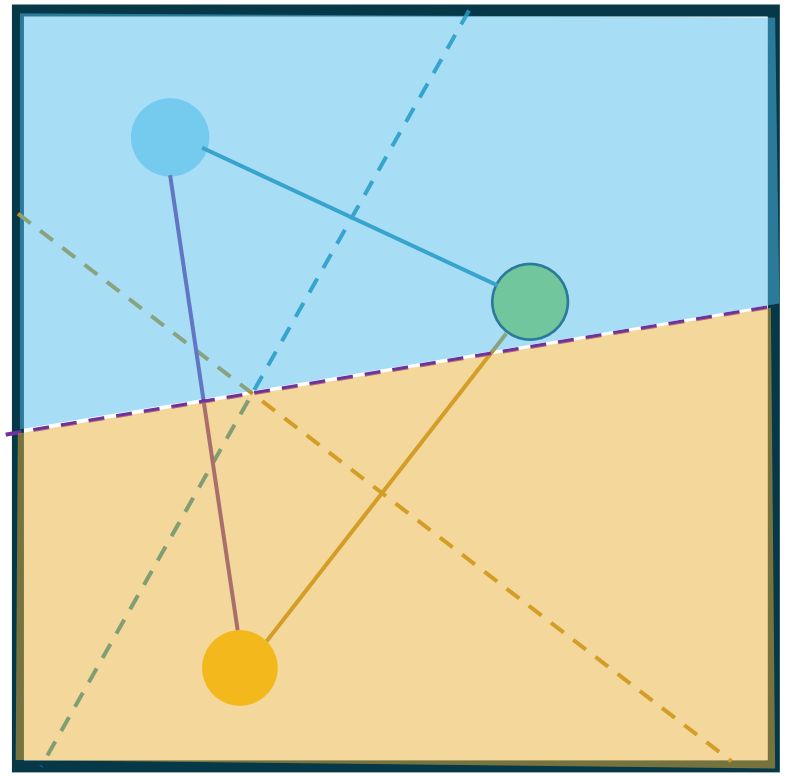
Let’s look at this geometrically



Let’s look at this geometrically

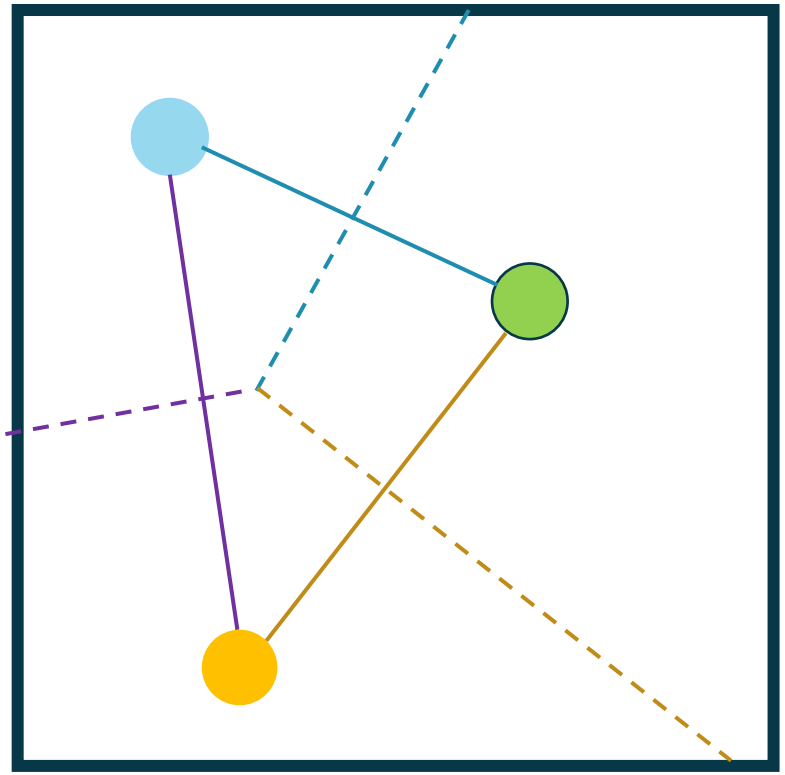


Let’s look at this geometrically



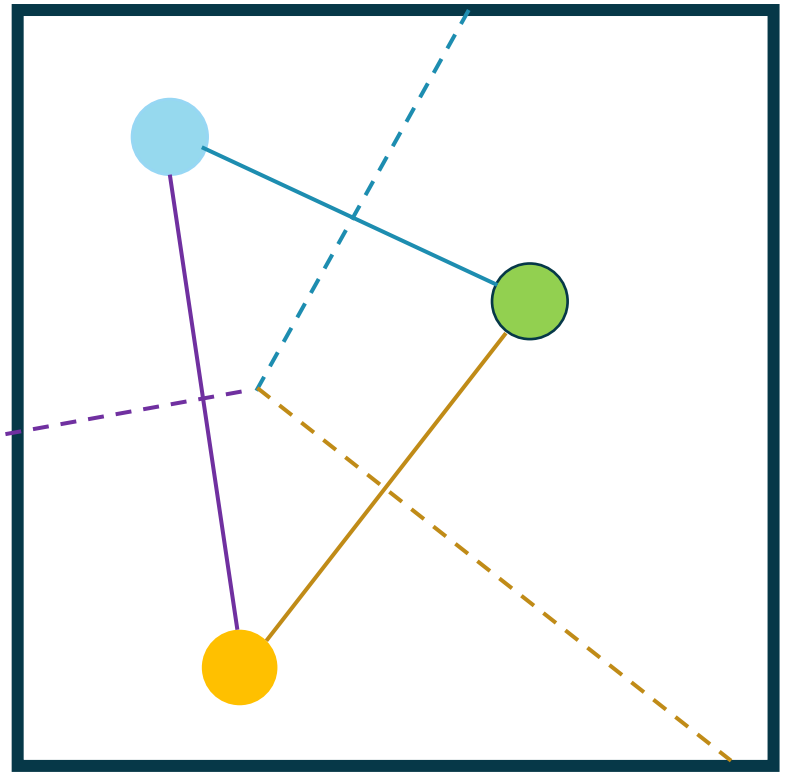
Let’s look at this geometrically

Voronoi diagram

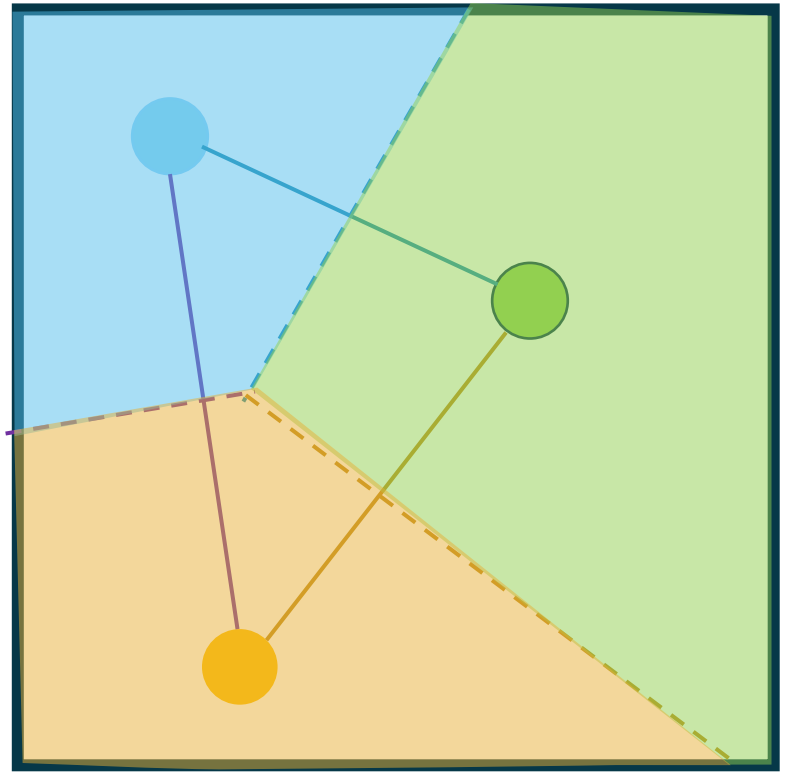


Let’s look at this geometrically

Voronoi diagram

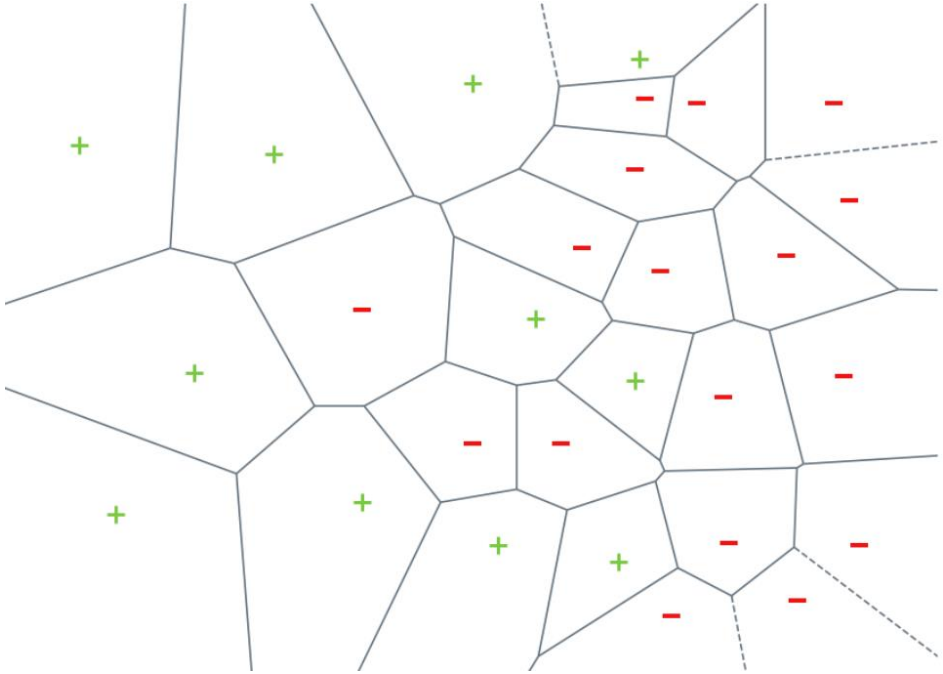


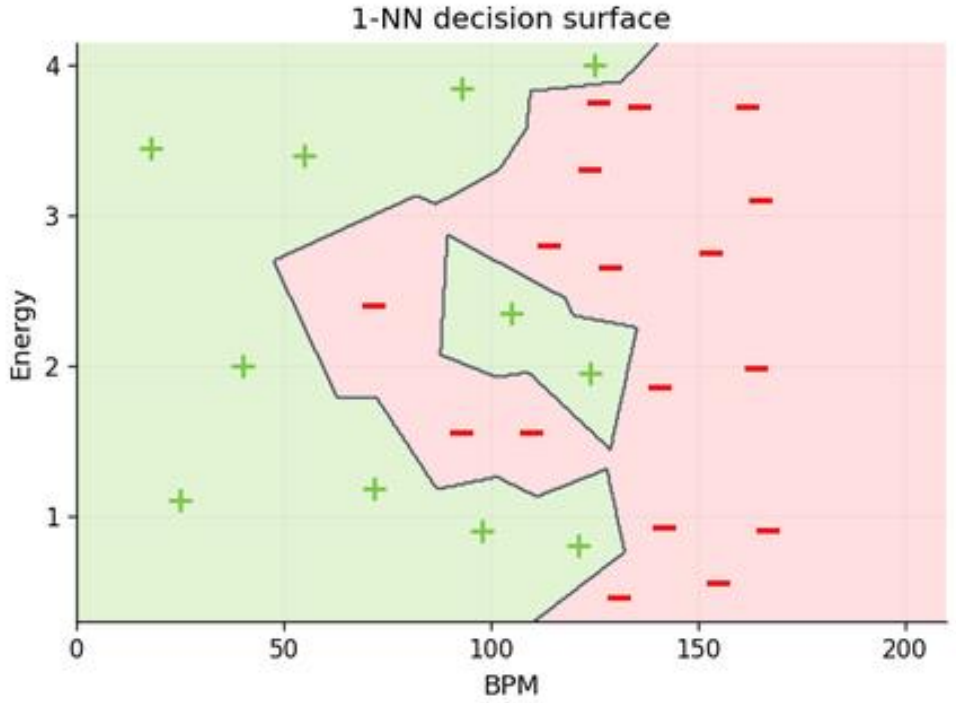
Decision surface for 1-NN



From voronoi diagrams to decision surfaces

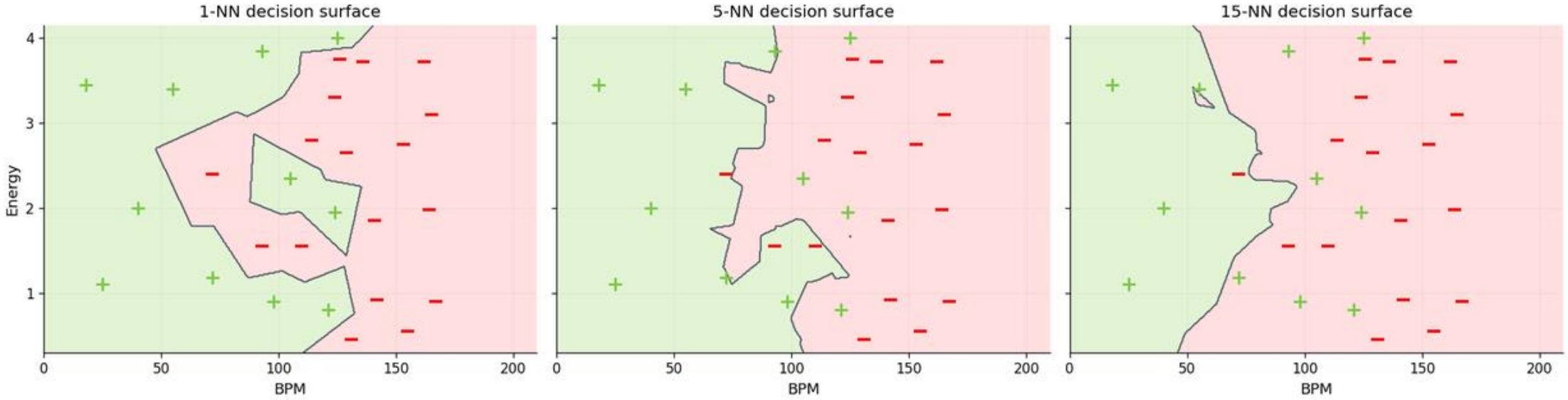
- Voronoi diagram: neighboring cells may make the same prediction
- Decision surface: separates regions that make different predictions





Effect of varying value of k

- Small k: each example can have a large effect on prediction → scattered surfaces → risk of overfitting
- Large k: averaging effect so each example has a more limited effect → smoother surfaces → risk of underfitting

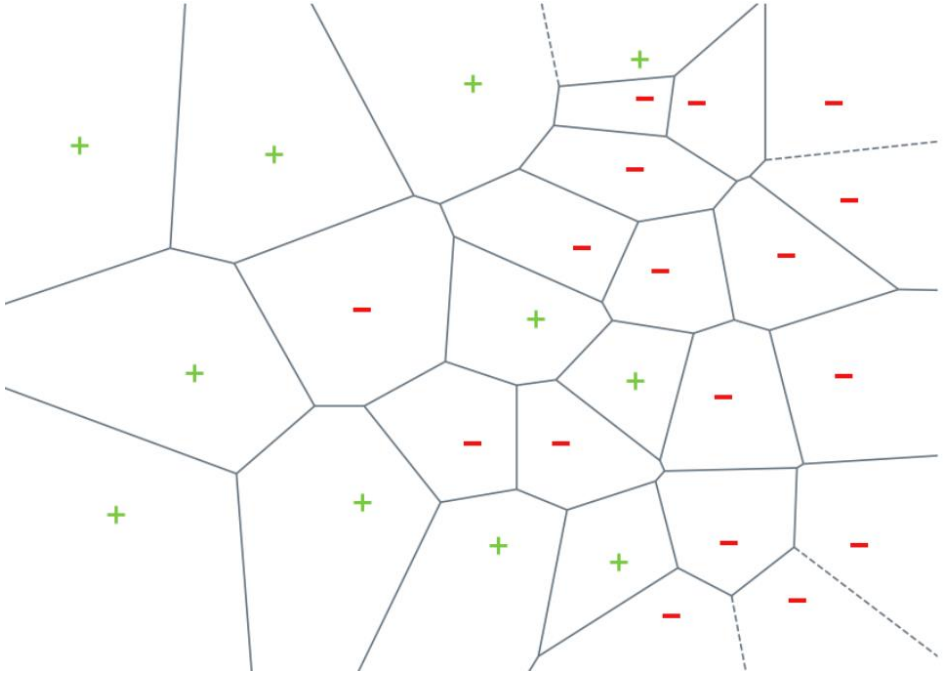


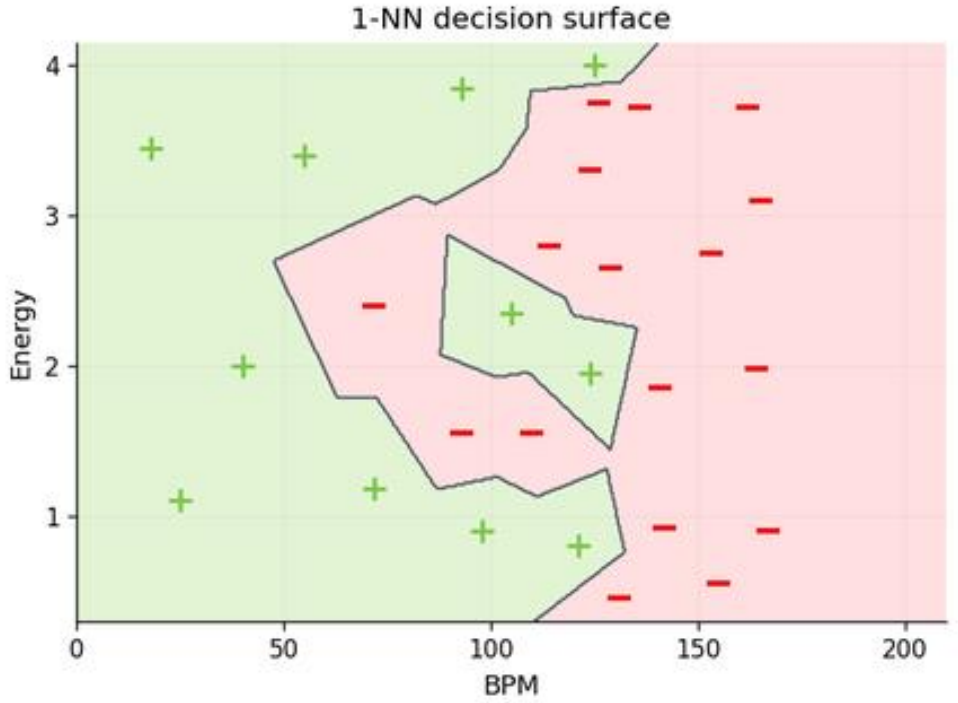
How to choose the parameters?

- Iterate over different ‘k’ values, different distance measures, …
- Use the training set as neighbors
- Use the validation set to find the best ‘k’, best distance measure, …
- Use the test set to evaluate against other models

What should we worry about when using kNN?

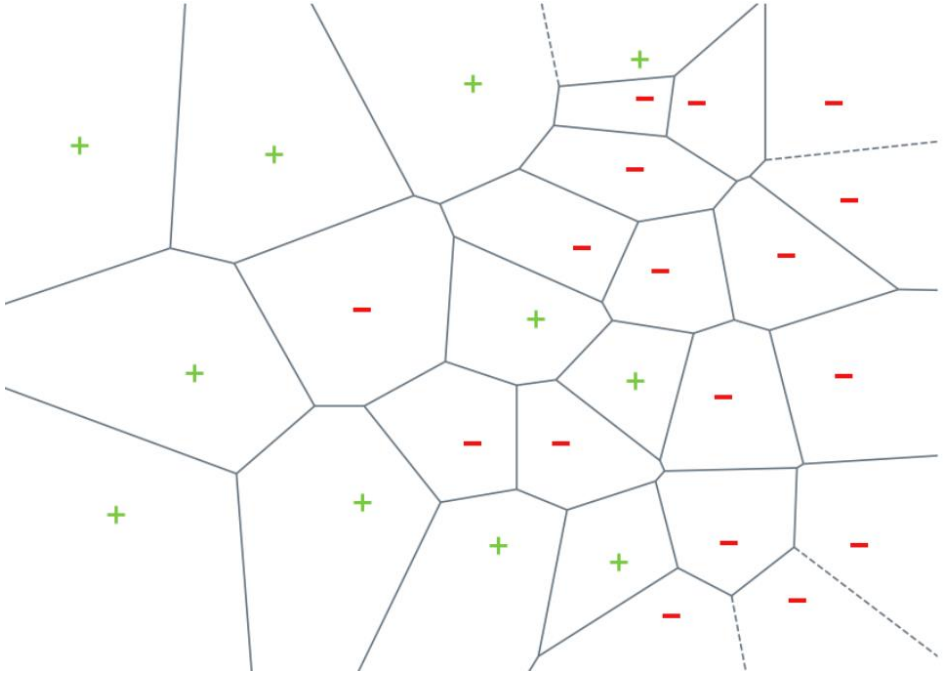
Is this a correct example?

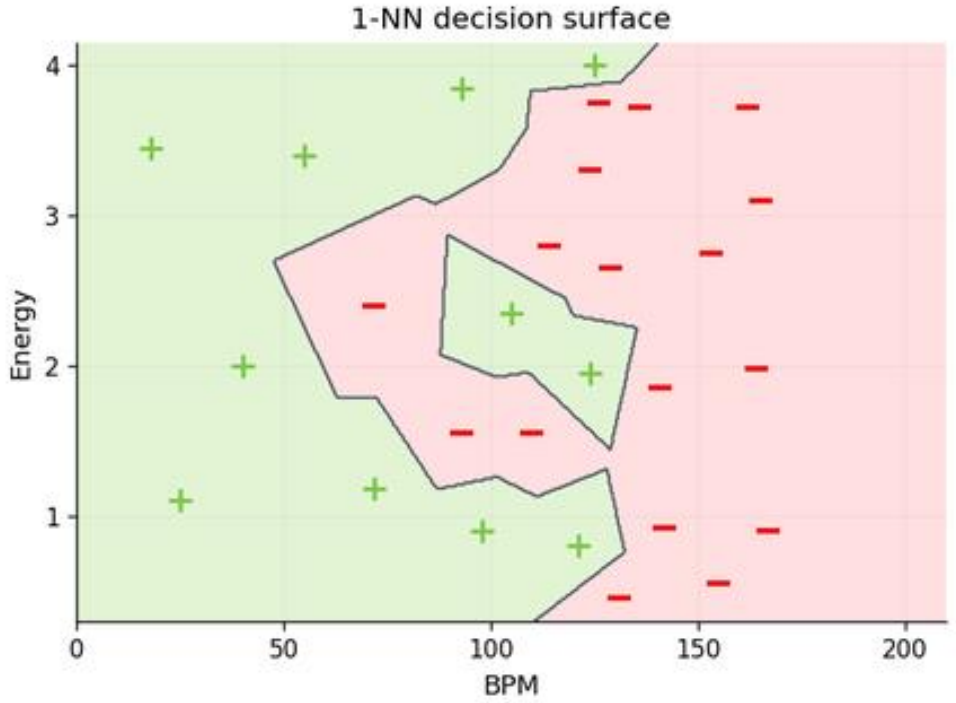




Problem 1: distances for multiple scales

- Which distance is used in this case?
- BPM and Energy have a different domain / scale (BPM higher than Energy)





Solution: Normalization

- Min-max normalization: Map between 0 and 1 → $$x_{i,k} = \frac{|x_{i,k} - \min_j x_{j,k}|}{\max_j x_{j,k} - \min_j x_{j,k}}$$

BPM	Energy
50	1.5
170	3
175	2.5



BPM	Energy
0	0
0.96	1
1	0.66

Solution: Normalization

- Min-max normalization: Map between 0 and 1 → $x_{i,k} = \frac{|x_{i,k} - \min_j x_{j,k}|}{\max_j x_{j,k} - \min_j x_{j,k}}$

BPM	Energy
50	1.5
170	3
175	2.5



BPM	Energy
0	0
0.96	1
1	0.66

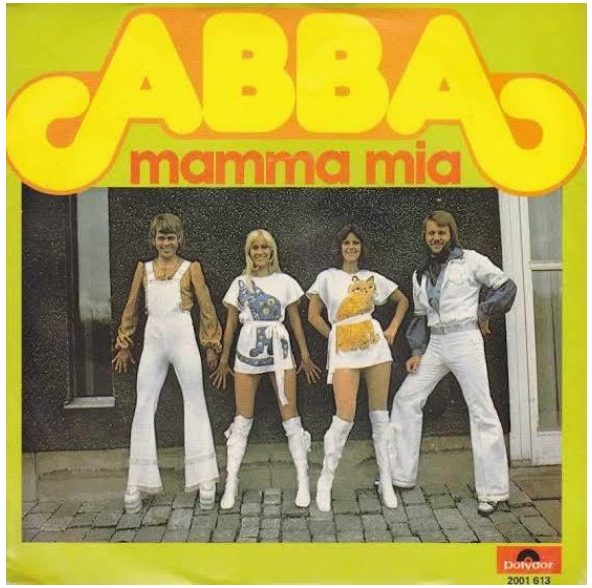
- Standardize each attribute value → $x_{i,k} = \frac{x_{i,k} - \mu_k}{\sigma_k}$

BPM	Energy
50	1.5
170	3
175	2.5



BPM	Energy
-1.413	-1.336
0.663	1.069
0.75	0.267

Problem 2: Illustration

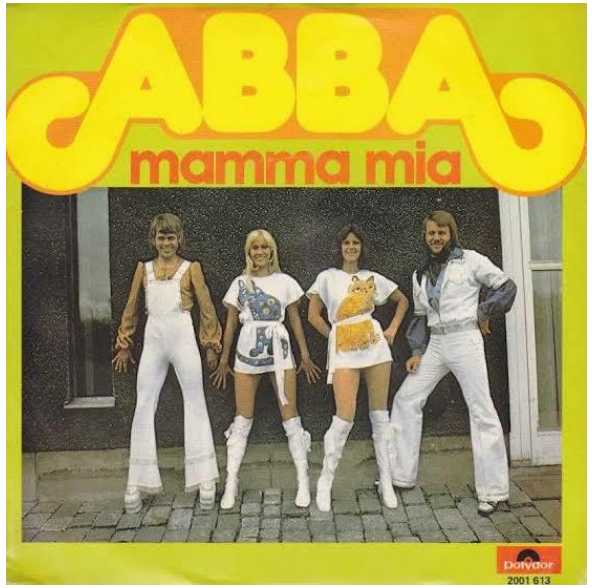


Duration	Key	BPM	Class
210s	D Major	137	Disco-pop



Duration	Key	BPM	Class
174s	A-flat major	186	Punk

Problem 2: Illustration



Duration	Key	BPM	Class
210s	D Major	137	Disco-pop

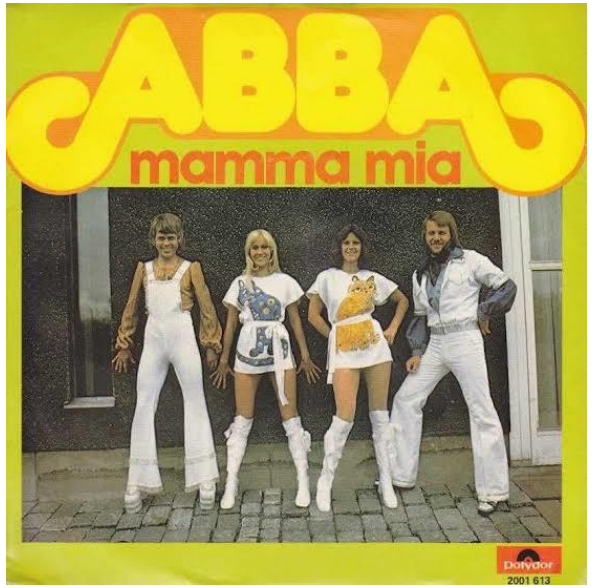


Duration	Key	BPM	Class
174s	A-flat major	160	?



Duration	Key	BPM	Class
174s	A-flat major	186	Punk

Problem 2: Illustration



Duration	Key	BPM	Class
210s	D Major	137	Disco-pop

Hamming distance = 2

Duration	Key	BPM	Class
174s	A-flat major	160	**PUNK**

Hamming distance = 1

Duration	Key	BPM	Class
174s	A-flat major	186	Punk





Problem 2: Irrelevant features

- Irrelevant features dominate distance

- Solutions
  - Carefully think about the feature space → Domain knowledge
  - Weigh the features in some way
  - Reduce the dimensionality → Automatically select only relevant features

Problem 3: Curse of dimensionality

- Weird things happen in high dimensions because our intuitions from low-dimensions do not translate
- Generally, adding dimensions (features) creates many problems for learners
- One big problem: with lots of features, one can end up with spurious correlations
- kNNs are easily misled with high-dimensional spaces
  - Density lowers → distances flatten and all examples become equidistant
  - Not the case when the distribution of positives has low variance

Problem 4

- kNN is very slow
  - Complexity is O(nd) with n the number of examples and d the dimension
  - All training examples should be in memory

- Solutions (see course on Big Data Analytics Programming for more details)
  - Efficient retrieval structures (KD trees, …)
  - Locality sensitive hashing (use hashing to identify neighbors)
  - Product quantization (Compress data, approximate distance based on embedding)

Pros and Cons of kNN

- Advantages
  - Training is fast
  - Learns complex target functions easily
  - Can predict discrete, real values, probabilities (dependent on distance measure)
  - Online or incremental learning is possible
  - Shares elements of human reasoning

- Disadvantages
  - Test time is very slow
  - Large memory cost
  - No explicit model to inspect

Supervised learning with kNN

Data: (X,Y) examples

→ **Representation:** choose features

→ **Inductive bias:** nearby examples have similar outputs

→ **Model/learner:** lazy, instance-based

→ **Choices:** *k,* distance measure, feature scaling/weighting

→ **Prediction:** find *k* nearest examples + combine labels

→ **Evaluation:** validation/test performance

→ **Pitfalls:** scaling, irrelevant features, dimensionality, computational cost

Questions?